DRAFT INTERNATIONAL STANDARD

ISO/DIS 26262-6

ISO/TC **22**/SC **32** | Secretariat: **JISC**

Voting begins on: **2016-09-21** | Voting terminates on: **2016-12-13**

Road vehicles — Functional safety —

Part 6: Product development at the software level

Véhicules routiers — Sécurité fonctionnelle —

Partie 6: Développement du produit au niveau du logiciel

ICS: 43.040.10

THIS DOCUMENT IS A DRAFT CIRCULATED FOR COMMENT AND APPROVAL. IT IS THEREFORE SUBJECT TO CHANGE AND MAY NOT BE REFERRED TO AS AN INTERNATIONAL STANDARD UNTIL PUBLISHED AS SUCH.

IN ADDITION TO THEIR EVALUATION AS BEING ACCEPTABLE FOR INDUSTRIAL, TECHNOLOGICAL, COMMERCIAL AND USER PURPOSES, DRAFT INTERNATIONAL STANDARDS MAY ON OCCASION HAVE TO BE CONSIDERED IN THE LIGHT OF THEIR POTENTIAL TO BECOME STANDARDS TO WHICH REFERENCE MAY BE MADE IN NATIONAL REGULATIONS.

RECIPIENTS OF THIS DRAFT ARE INVITED TO SUBMIT, WITH THEIR COMMENTS, NOTIFICATION OF ANY RELEVANT PATENT RIGHTS OF WHICH THEY ARE AWARE AND TO PROVIDE SUPPORTING DOCUMENTATION.

This document is circulated as received from the committee secretariat.

Reference number
ISO/DIS 26262-6:2016(E)

Licensed to MDS Technology CO., LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-146344 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

© ISO 2016

COPYRIGHT PROTECTED DOCUMENT

© ISO 2016, Published in Switzerland

All rights reserved. Unless otherwise specified, no part of this publication may be reproduced or utilized otherwise in any form or by any means, electronic or mechanical, including photocopying, or posting on the internet or an intranet, without prior written permission. Permission can be requested from either ISO at the address below or ISO's member body in the country of the requester.

ISO copyright office
Ch. de Blandonnet 8 • CP 401
CH-1214 Vernier, Geneva, Switzerland
Tel. +41 22 749 01 11
Fax +41 22 749 09 47
copyright@iso.org
www.iso.org

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

© ISO 2016 – All rights reserved

Contents

Foreword 6

Introduction 7

1 Scope 9

2 Normative references 10

3 Terms and definitions 10

4 Requirements for compliance 10

5 General topics for the product development at the software level 12

5.1 Objectives 12
5.2 General 12
5.3 Inputs to this clause 13
5.3.1 Prerequisites 13
5.3.2 Further supporting information 13
5.4 Requirements and recommendations 14
5.5 Work products 15

6 Specification of software safety requirements 16

6.1 Objectives 16
6.2 General 16
6.3 Inputs to this clause 16
6.3.1 Prerequisites 16
6.3.2 Further supporting information 16
6.4 Requirements and recommendations 16
6.5 Work products 18

7 Software architectural design 18

7.1 Objectives 18
7.2 General 19
7.3 Inputs to this clause 19
7.3.1 Prerequisites 19
7.3.2 Further supporting information 19
7.4 Requirements and recommendations 19
7.5 Work products 26

8 Software unit design and implementation 26

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

© ISO 2016 – All rights reserved

8.1 Objectives 26
8.2 General 26
8.3 Inputs to this clause 27
8.3.1 Prerequisites 27
8.3.2 Further supporting information 27
8.4 Requirements and recommendations 27
8.5 Work products 29
9 Software unit verification 29
9.1 Objectives 29
9.2 General 29
9.3 Inputs to this clause 29
9.3.1 Prerequisites 29
9.3.2 Further supporting information 30
9.4 Requirements and recommendations 30
9.5 Work products 33
10 Software integration and verification 33
10.1 Objectives 33
10.2 General 33
10.3 Inputs to this clause 34
10.3.1 Prerequisites 34
10.3.2 Further supporting information 34
10.4 Requirements and recommendations 34
10.5 Work products 37
11 Testing of the embedded software 37
11.1 Objective 37
11.2 General 37
11.3 Inputs to this clause 38
11.3.1 Prerequisites 38
11.3.2 Further supporting information 38
11.4 Requirements and recommendations 38
11.5 Work products 39
Annex A (informative) Overview of and workflow of management of product development at the software level 40
Annex B (informative) Model-based development approaches 42

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

© ISO 2016 – All rights reserved

Annex C (normative) Software configuration **46**

Annex D (informative) Freedom from interference between software elements **53**

Annex E (informative) Application of safety analyses and analyses of dependent failures at the software architectural level **56**

E.1 Objectives 56

E.2 General 56

E.3 Topics for performing analyses 60

E.4 Mitigation strategy for weaknesses identified during safety and dependent failure analyses 67

Bibliography **70**

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

© ISO 2016 – All rights reserved

Foreword

ISO (the International Organization for Standardization) is a worldwide federation of national standards bodies (ISO member bodies). The work of preparing International Standards is normally carried out through ISO technical committees. Each member body interested in a subject for which a technical committee has been established has the right to be represented on that committee. International organizations, governmental and non-governmental, in liaison with ISO, also take part in the work. ISO collaborates closely with the International Electrotechnical Commission (IEC) on all matters of electrotechnical standardization.

The procedures used to develop this document and those intended for its further maintenance are described in the ISO/IEC Directives, Part 1. In particular the different approval criteria needed for the different types of ISO documents should be noted. This document was drafted in accordance with the editorial rules of the ISO/IEC Directives, Part 2. www.iso.org/directives

Attention is drawn to the possibility that some of the elements of this document may be the subject of patent rights. ISO shall not be held responsible for identifying any or all such patent rights. Details of any patent rights identified during the development of the document will be in the Introduction and/or on the ISO list of patent declarations received. www.iso.org/patents

Any trade name used in this document is information given for the convenience of users and does not constitute an endorsement.

For an explanation on the meaning of ISO specific terms and expressions related to conformity assessment, as well as information about ISO's adherence to the WTO principles in the Technical Barriers to Trade (TBT) see the following URL: Foreword - Supplementary information

The committee responsible for this document is ISO/TC22/SC32 Electrical and electronic components and general system aspects.

This second edition cancels and replaces the first edition which has been technically revised.

A list of all parts in the ISO 26262 series can be found on the ISO website.

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

Introduction

ISO 26262 is the adaptation of IEC 61508 to address the sector specific needs of electrical and/or electronic (E/E) systems within road vehicles.

This adaptation applies to all activities during the safety lifecycle of safety-related systems comprised of electrical, electronic and software components.

Safety is one of the key issues in the development of road vehicles. Development and integration of automotive functionalities strengthen the need for functional safety and the need to provide evidence that functional safety objectives are satisfied.

With the trend of increasing technological complexity, software content and mechatronic implementation, among others there are increasing risks from systematic failures and random hardware failures, these being considered within the scope of functional safety. ISO 26262 includes guidance to mitigate these risks by providing appropriate requirements and processes.

To achieve functional safety, ISO 26262:

a) provides a reference for the automotive safety lifecycle and supports the tailoring of the activities to be performed during the lifecycle phases, i.e., development, production, operation, service, and decommissioning;
b) provides an automotive-specific risk-based approach to determine integrity levels [Automotive Safety Integrity Levels (ASIL)];
c) uses ASILs to specify which of the requirements of ISO 26262 are applicable to avoid unreasonable residual risk;
d) provides requirements for functional safety management, verification, validation and confirmation measures; and
e) provides requirements for relations with suppliers.

ISO 26262 is concerned with functional safety of E/E systems that is achieved through safety measures including safety mechanisms. It also provides a framework within which safety-related systems based on other technologies (e.g. mechanical, hydraulic and pneumatic) can be considered.

The achievement of functional safety is influenced by the development process (including such activities as requirements specification, design, implementation, integration, verification, validation and configuration), the production and service processes and the management processes.

Safety is intertwined with common function-oriented and quality-oriented activities and work products. ISO 26262 addresses the safety-related aspects of these activities and work products.

Figure 1 shows the overall structure of ISO 26262. ISO 26262 is based upon a V-model as a reference process model for the different phases of product development. Within the figure:

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

— the shaded "V"s represent the interconnection among ISO 26262-3, ISO 26262-4, ISO 26262-5, ISO 26262-6 and ISO 26262-7; for motorcycles ISO 26262-12 clauses 6 supplements ISO 26262-3 and clause 7, 8 and 9 supplements Part 4.

— the specific clauses are indicated in the following manner: "m-n", where "m" represents the number of the particular part and "n" indicates the number of the clause within that part.

EXAMPLE "2-6" represents Clause 6 of ISO 26262-2.

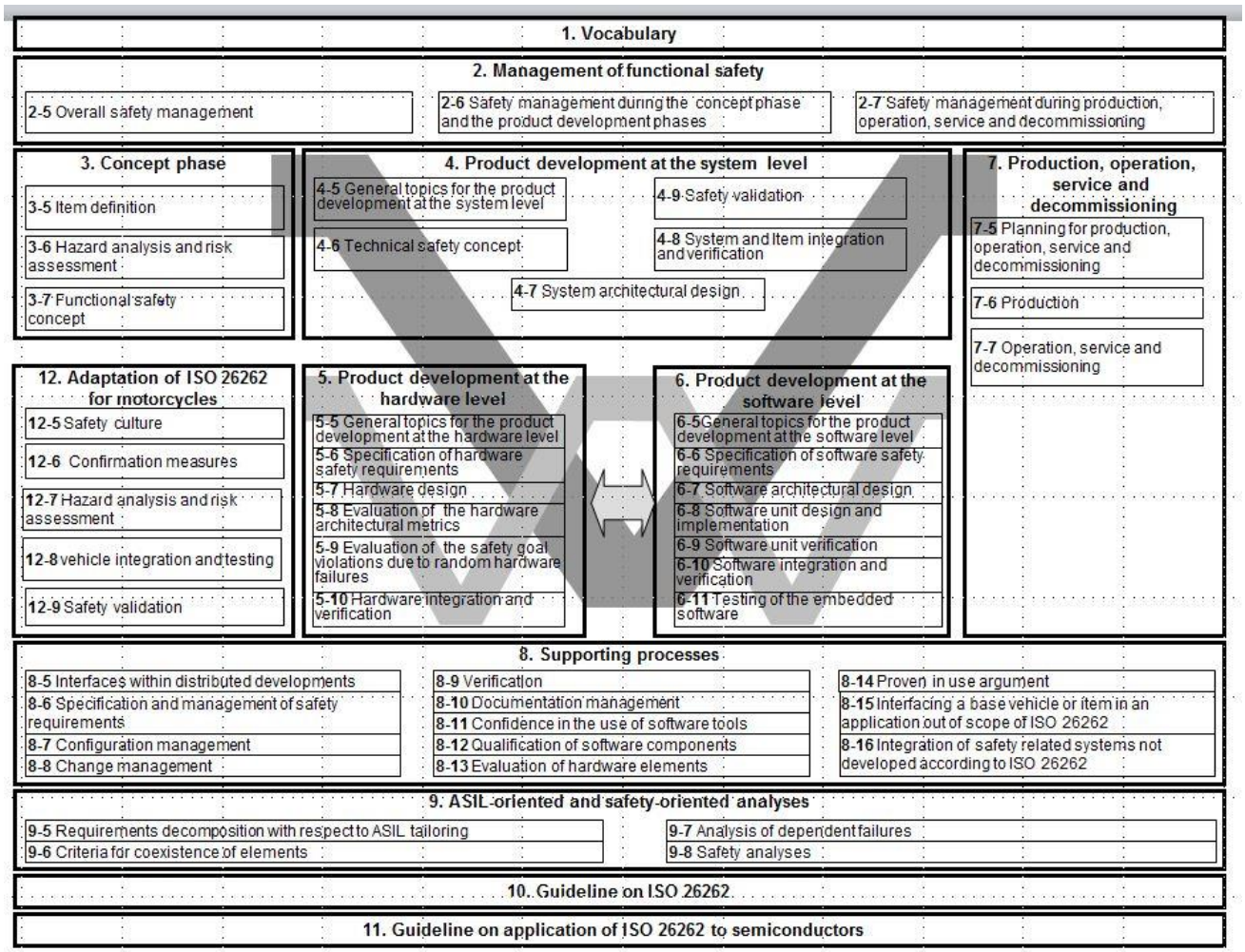


Figure 1 — Overview of ISO 26262

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

Road Vehicles — Functional Safety — Part 6: Product development at the software level

1 Scope

ISO 26262 is intended to be applied to safety-related systems that include one or more electrical and/or electronic (E/E) systems and that are installed in series production road vehicles, excluding mopeds. ISO 26262 does not address unique E/E systems in special vehicles such as E/E systems designed for drivers with disabilities.

NOTE Other dedicated application-specific safety standards exist and may complement ISO 26262 or vice versa.

Systems and their components released for production, or systems and their components already under development prior to the publication date of this edition of ISO 26262, are exempted from the scope of this edition. For further development or alterations based on systems and their components released for production prior to the publication of this edition of ISO 26262, only the modifications will be developed in accordance with this edition of ISO 26262. This edition of ISO 26262 addresses integration of existing systems not developed according to this edition of ISO 26262 and systems developed according to this edition of ISO 26262 by tailoring the safety lifecycle.

ISO 26262 addresses possible hazards caused by malfunctioning behaviour of safety-related E/E systems, including interaction of these systems. It does not address hazards related to electric shock, fire, smoke, heat, radiation, toxicity, flammability, reactivity, corrosion, release of energy and similar hazards, unless directly caused by malfunctioning behaviour of safety-related E/E systems.

ISO 26262 does not address the nominal performance of E/E systems, even if functional performance standards exist for these systems (e.g. active and passive safety systems, brake systems, adaptive cruise control).

ISO 26262 describes a framework for functional safety to assist the development of safety-related E/E systems. This framework is intended to be used to integrate functional safety activities into a company-specific development framework. Some requirements have a clear technical focus to implement functional safety into a product; others address the development process and can therefore be seen as process requirements in order to demonstrate the capability of an organization with respect to functional safety.

This part of ISO 26262 specifies the requirements for product development at the software level for automotive applications, including the following:

— general topics for product development at the software level;

— specification of the software safety requirements;

— software architectural design;

— software unit design and implementation;

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

— software unit verification;

— software integration and verification; and

— testing of the embedded software.

It also specifies requirements associated with the use of configurable software.

2 Normative references

The following documents, in whole or in part, are normatively referenced in this document and are indispensable for its application. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

ISO 26262-1:2018, *Road Vehicles — Functional Safety— Part 1: Vocabulary*

ISO 26262-2:2018, *Road Vehicles — Functional Safety— Part 2: Management of functional safety*

ISO 26262-3:2018, *Road vehicles — Functional safety — Part 3: Concept phase*

ISO 26262-4:2018, *Road vehicles — Functional safety — Part 4: Product development at the system level*

ISO 26262-5:2018, *Road vehicles — Functional safety — Part 5: Product development at the hardware level*

ISO 26262-7:2018, *Road vehicles — Functional safety — Part 7: Production, operation, service and decommissioning*

ISO 26262-8:2018, *Road vehicles — Functional safety — Part 8: Supporting processes*

ISO 26262-9:2018, *Road vehicles — Functional safety — Part 9: Automotive Safety Integrity Level (ASIL)-oriented and safety-oriented analyses*

3 Terms and definitions

For the purposes of this document, the terms, definitions and abbreviated terms given in ISO 26262-1:2018 apply.

4 Requirements for compliance

4.1 Purpose

This clause describes how:

a) to achieve compliance with ISO 26262;

b) to interpret the tables used in ISO 26262; and

c) to interpret the applicability of each clause, depending on the relevant ASIL(s).

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

4.2 General requirements

When claiming compliance with ISO 26262, each requirement shall be met, unless one of the following applies:

a) tailoring of the safety activities in accordance with ISO 26262-2 has been performed that shows that the requirement does not apply, or

b) a rationale is available that the non-compliance is acceptable and the rationale has been assessed in accordance with ISO 26262-2, for example as part of the functional safety assessment.

Informative content, including notes and examples, is only for guidance in understanding, or for clarification of the associated requirement, and shall not be interpreted as a requirement itself or as complete or exhaustive.

The results of safety activities are given as work products. "Prerequisites" are information which shall be available as work products of a previous phase. Given that certain requirements of a clause are ASIL-dependent or may be tailored, certain work products may not be needed as prerequisites.

"Further supporting information" is information that can be considered, but which in some cases is not required by ISO 26262 as a work product of a previous phase and which may be made available by external sources that are different from the persons or organizations responsible for the functional safety activities.

4.3 Interpretations of tables

Tables are normative or informative depending on their context. The different methods listed in a table contribute to the level of confidence in achieving compliance with the corresponding requirement. Each method in a table is either

a) a consecutive entry (marked by a sequence number in the leftmost column, e.g. 1, 2, 3), or

b) an alternative entry (marked by a number followed by a letter in the leftmost column, e.g. 2a, 2b, 2c).

For consecutive entries, all listed highly recommended and recommended methods in accordance with the ASIL apply. It is allowed to substitute a highly recommended or recommended method by other one(s) not listed in the table, but a rationale shall be given that these comply with the corresponding requirement. A recommended method may be omitted, but a rationale why this method is omitted shall be given.

For alternative entries, an appropriate combination of methods shall be applied in accordance with the ASIL indicated, independent of whether they are listed in the table or not. If methods are listed with different degrees of recommendation for an ASIL, the methods with the higher recommendation should be preferred. A rationale shall be given that the selected combination of methods or even a selected single method complies with the corresponding requirement.

NOTE A rationale based on the methods listed in the table is sufficient. However, this does not imply a bias for or against methods not listed in the table.

For each method, the degree of recommendation to use the corresponding method depends on the ASIL and is categorized as follows:

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

— "++" indicates that the method is highly recommended for the identified ASIL;

— "+" indicates that the method is recommended for the identified ASIL;

— "o" indicates that the method has no recommendation for or against its usage for the identified ASIL.

4.4 ASIL-dependent requirements and recommendations

The requirements or recommendations of each subclause shall be met for ASIL A, B, C and D, if not stated otherwise. These requirements and recommendations refer to the ASIL of the safety goal. If ASIL decomposition has been performed at an earlier stage of development, in accordance with ISO 26262-9:2018, Clause 5, the ASIL resulting from the decomposition shall be met.

If an ASIL is given in parentheses in ISO 26262, the corresponding subclause shall be considered as a recommendation rather than a requirement for this ASIL. This has no link with the parenthesis notation related to ASIL decomposition.

4.5 Adaptation for motorcycles

For items or elements for which requirements of ISO 26262-12 are applicable, the requirements of ISO 26262-12 supersede the corresponding requirements in this Part of ISO 26262.

4.6 Adaptation for Trucks, Buses, Trailers and Semitrailers

Content that is intended to be unique for Trucks, Buses, Trailers and Semitrailers (T&B) is indicated as such.

5 General topics for the product development at the software level

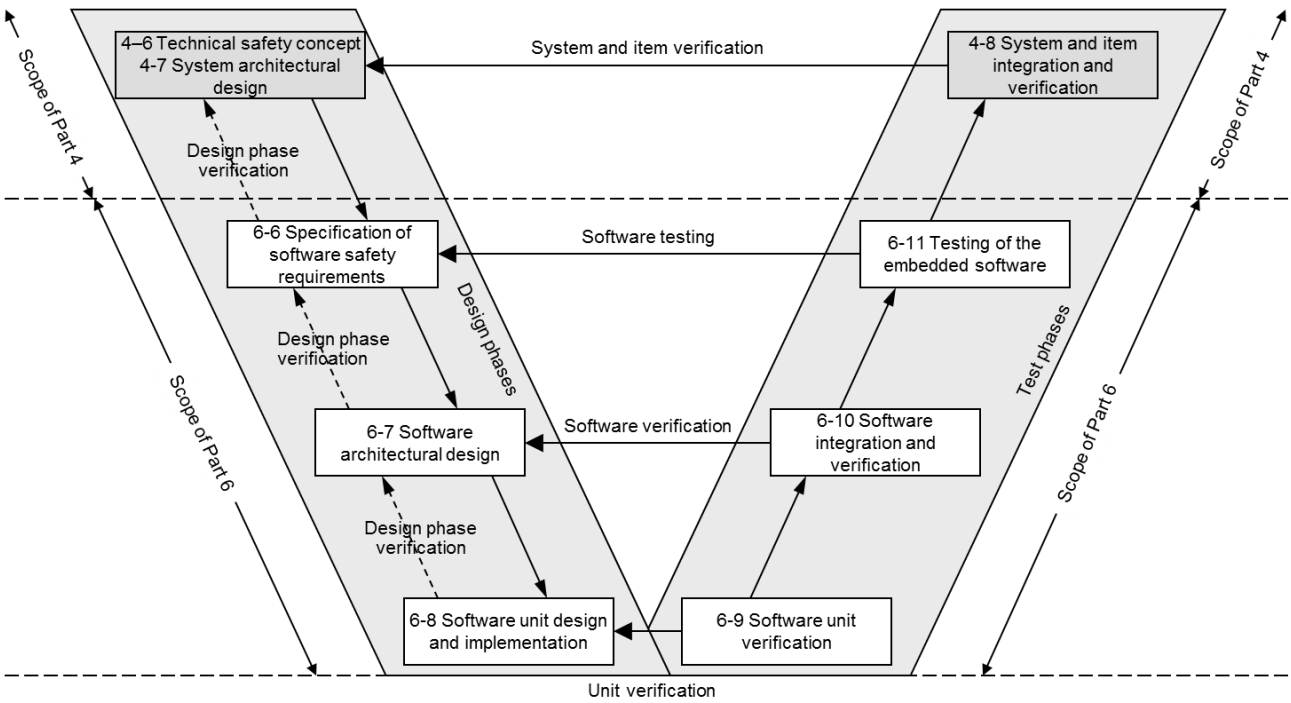
5.1 Objectives

The objective of this clause is to give an overview of the product development at the software level and to provide topics specific to software development.

5.2 General

The reference phase model for the development of software is given in Figure 2. Details concerning the treatment of configurable software are provided in Annex C.

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.



NOTE Within the figure, the specific clauses of each part of ISO 26262 are indicated in the following manner: "m-n", where "m" represents the number of the part and "n" indicates the number of the clause, e.g. "4-7" represents Clause 7 of ISO 26262-4.

Figure 2 — Reference phase model for the software development

NOTE Methods from Agile Software Development may also be suitable for the development of safety-related software but Agile processes cannot be used to avoid requirements for documentation, process or product rigour.

EXAMPLE Improve quality and testability of requirements by Test Driven Development.

In order to be able to develop software, specific topics are addressed in this clause concerning the modelling and/or programming languages to be used, and the application of guidelines and tools.

NOTE Tools used for software development can include tools other than software tools.

EXAMPLE Tools used for testing phases.

5.3 Inputs to this clause

5.3.1 Prerequisites

The following information shall be available:

— (none)

5.3.2 Further supporting information

The following information can be considered:

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

— qualified software tools available (see ISO 26262-8:2018, Clause 11);

— design and coding guidelines for modelling and programming languages (from an external source);

— guidelines for the application of methods (from an external source); and

— guidelines for the application of tools (from an external source).

5.4 Requirements and recommendations

5.4.1 The software development process for the software of an item, including lifecycle phases, methods, languages and tools, shall be consistent across all the subphases of the software lifecycle and be compatible with the system and hardware development phases, such that the required data can be used consistently between phases.

NOTE 1 The sequencing of phases, tasks and activities, including iteration steps, for the software of an item intends to ensure the consistency of the corresponding work products with the product development at the hardware level (see ISO 26262-5) and the product development at the system level (see ISO 26262-4).

EXAMPLE Agile methods such as continuous integration based on an automated build system can support consistency of subphases and facilitate regression tests. Such a build system typically performs code generation, compiling and linking, static and semantic code analysis, documentation generation, testing and packaging. It allows, subject to tool chain and tool configuration, repeatable and, after changes, comparable production of software, documentation and test results. However the required process rigour cannot be ignored.

NOTE 2 The software tool criteria evaluation report (see ISO 26262-8, 11.5.1) or the software tool qualification report (see ISO 26262-8, 11.5.2) can provide input to the tool application guidelines.

5.4.2 The criteria that shall be considered when selecting a suitable modelling or programming language are:

a) an unambiguous and understandable definition;

EXAMPLE Unambiguous definition of syntax and semantics or restriction to configuration of the development environment.

b) suitability for the design or/and implementation of embedded real time software including runtime error handling;

c) support the achievement of modularity and abstraction;

d) support the use of structured constructs; and

e) suitability for specifying and managing safety requirements according to ISO 26262-8 Clause 6, if modelling is used for requirements engineering and management.

NOTE 2 Assembly languages can be used for those parts of the software where the use of high-level programming languages is not appropriate, such as low-level software with interfaces to the hardware, interrupt handlers, or time-critical algorithms.

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

5.4.3 Criteria for suitable modelling or programming languages (see 5.4.2) that are not sufficiently addressed by the language itself shall be covered by the corresponding guidelines, or by the development environment, considering the topics listed in Table 1.

EXAMPLE 1 MISRA C[3] is a coding guideline for the programming language C and includes guidance for automatically generated code.

EXAMPLE 2 In the case of model based development with automatic code generation, guidelines can be applied at the model level as well as the code level. Appropriate modelling style guides including MISRA AC series can be considered. Style guides for commercial tools are also possible guidelines.

NOTE 3 Existing coding guidelines and modelling guidelines can be modified for a specific item development.

Table 1 — Topics to be covered by modelling and coding guidelines

Topics		ASIL			
		A	B	C	D
1a	Enforcement of low complexity [a]	++	++	++	++
1b	Use of language subsets [b]	++	++	++	++
1c	Enforcement of strong typing [c]	++	++	++	++
1d	Use of defensive implementation techniques [d]	+	+	++	++
1e	Use of well-trusted design principles [e]	+	+	++	++
1f	Use of unambiguous graphical representation	+	++	++	++
1g	Use of style guides	+	++	++	++
1h	Use of naming conventions	++	++	++	++
1i	Representation of concurrency aspects [f]	+	+	+	+

[a] An appropriate compromise of this topic with other requirements of this part of ISO 26262 may be required.

[b] The objectives of topic 1b include:

— Exclusion of ambiguously-defined language constructs which may be interpreted differently by different modellers, programmers, code generators or compilers.
— Exclusion of language constructs which from experience easily lead to mistakes, for example assignments in conditions or identical naming of local and global variables.
— Exclusion of language constructs which could result in unhandled run-time errors.

[c] The objective of topic 1c is to impose principles of strong typing where these are not inherent in the language.

[d] Examples of defensive implementation techniques:

— Verify the divisor before a division operation (different from zero or in a specific range).
— Check an identifier passed by parameter to verify that the calling function is the intended caller.
— Use the "default" in switch cases to detect an error.

[e] Verification of the validity of the underlying assumptions, boundaries and conditions of application may be required.

[f] Concurrency of processes or tasks may be a topic when executing software in a multi-core or multi-processor runtime environment

5.5 Work products

5.5.1 Design and coding guidelines for modelling and programming languages resulting from requirements 5.4.2 and 5.4.3.

5.5.2 Tool application guidelines resulting from requirements 5.4.1 to 5.4.3.

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

6 Specification of software safety requirements

6.1 Objectives

The objectives of this subphase are:

a) to specify the software safety requirements. They are derived from the technical safety concept and the system design specification;

b) to refine the hardware-software interface requirements initiated in ISO 26262-4:2018, Clause 7; and

c) to verify that the software safety requirements and the hardware-software interface requirements are consistent with the technical safety concept and the system design specification.

6.2 General

The technical safety requirements are refined and allocated to hardware and software during the system architectural design phase given in ISO 26262-4:2018, Clause 7. The specification of the software safety requirements considers in particular constraints of the hardware and the impact of these constraints on the software. This subphase includes the specification of software safety requirements to support the subsequent design phases.

6.3 Inputs to this clause

6.3.1 Prerequisites

The following information shall be available:

— technical safety concept in accordance with ISO 26262-4:2018, 7.5.1;

— system architectural design specification in accordance with ISO 26262-4:2018, 7.5.2; and

— hardware-software interface (HSI) specification in accordance with ISO 26262-4:2018, 7.5.3.

6.3.2 Further supporting information

The following information can be considered:

— hardware design specification (see ISO 26262-5:2018, 7.5.1);

— specification of non-safety-related functions and properties of the software (from an external source); and

— guidelines for the application of methods (from an external source).

6.4 Requirements and recommendations

6.4.1 The software safety requirements shall be derived considering the required safety-related functionalities and properties of the software, whose failures could lead to violation of a technical safety requirement allocated to software.

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

NOTE 1 The software safety requirements are either derived directly from the technical safety requirements allocated to software, or are requirements for software functions and properties that if not fulfilled could lead to a violation of the technical safety requirements allocated to software.

EXAMPLE 1 Safety-related functionality of the software can be:

— functions that enable the safe execution of a nominal function,

— functions that enable the system to achieve or maintain a safe state or degraded state,

— functions related to the detection, indication and mitigation of faults of safety-related hardware elements,

— self-test or monitoring functions related to the detection, indication and mitigation failures in the operating system, basic software or the application software itself ,

— functions related to on-board and off-board tests during production, operation, service and decommissioning,

— functions that allow modifications of the software during production and service, or

— functions related to performance or time-critical operations.

EXAMPLE 2 Safety-related properties include robustness against erroneous inputs, independence or freedom from interference between different functionalities, fault tolerance capabilities of the software.

NOTE 2 Safety analyses can be used to identify additional software safety requirements or provide evidence for their achievement.

6.4.2 Specification of the software safety requirements derived from the technical safety concept and the system design in accordance with ISO 26262-4:2018, 7.4.1 and 7.4.5 shall consider:

a) the specification and management of safety requirements in accordance with ISO 26262-8:2018, Clause 6;

b) the specified system and hardware configurations;

EXAMPLE 1 Configuration parameters can include gain control, band pass frequency and clock prescaler.

c) the hardware-software interface specification;

d) the relevant requirements of the hardware design specification;

e) the timing constraints;

EXAMPLE 2 Execution or reaction time derived from the required response time at the system level.

f) the external interfaces; and

EXAMPLE 3 Communication and user interfaces.

g) each operating mode and each transition between the operating modes of the vehicle, the system, or the hardware, having an impact on the software.

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

EXAMPLE 4 Operating modes include shut-off or sleep, initialization, normal operation, degraded and advanced modes for testing or flash programming.

6.4.3 If ASIL decomposition is applied to the software safety requirements, ISO 26262-9:2018, Clause 5, shall be complied with.

6.4.4 The hardware-software interface specification initiated in ISO 26262-4:2018, Clause 7, shall be refined sufficiently to allow for the correct control and usage of the hardware by the software, and shall describe each safety-related dependency between hardware and software.

6.4.5 If other functions in addition to those functions for which safety requirements are specified in 6.4.1 are carried out by the embedded software, a specification of these functions and their properties in accordance with the applied quality management system shall be available.

6.4.6 The refined hardware-software interface specification shall be verified jointly by the persons responsible for the system, hardware and software development.

6.4.7 The software safety requirements and the refined requirements of the hardware-software interface specification shall be verified in accordance with ISO 26262-8:2018, Clauses 6 and 9, to provide evidence for their:

a) compliance and consistency with the technical safety requirements;

b) compliance with the system design; and

c) consistency with the hardware-software interface.

6.5 Work products

6.5.1 **Software safety requirements specification** resulting from requirements 6.4.1 to 6.4.3 and 6.4.5.

6.5.2 **Hardware-software interface (HSI) specification (refined)** resulting from requirement 6.4.4.

NOTE This work product refers to the same work product as given in ISO 26262-5:2018, 6.5.2

6.5.3 **Software verification report** resulting from requirements 6.4.6 and 6.4.7.

7 Software architectural design

7.1 Objectives

The objective of this subphase are:

a) to develop a software architectural design that satisfies the software safety requirements and the non-safety-related requirements;

b) to verify that the software architectural design is suitable to satisfy the software safety requirements with the required ASIL; and

c) to support the implementation of the software.

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

7.2 General

The software architectural design represents the software architectural elements and their interactions in a hierarchical structure. Static aspects, such as interfaces between the software components, as well as dynamic aspects, such as process sequences and timing behaviour are described.

NOTE The software architectural design is not necessarily limited to one microcontroller or ECU. The software architecture for each microcontroller is also addressed by this subclause.

A software architectural design must be able to satisfy both software safety requirements as well as the non-safety-related requirements. Hence in this subphase safety-related and non-safety-related requirements are handled within one development process.

The software architectural design provides the means to implement both the software requirements and the software safety requirements with the required integrity and to manage the complexity of the detailed design and the implementation of the software.

7.3 Inputs to this clause

7.3.1 Prerequisites

The following information shall be available:

— design and coding guidelines for modelling and programming languages in accordance with 5.5.1;

— hardware-software interface (HSI) specification (refined) in accordance with 6.5.2; and

— software safety requirements specification in accordance with 6.5.1.

7.3.2 Further supporting information

The following information can be considered:

— technical safety concept (see ISO 26262-4:2018, 7.5.1);

— system architectural design specification (see ISO 26262-4:2018, 7.5.2);

— qualified software components available (see ISO 26262-8:2018, Clause 12);

— tool application guidelines (see 5.5.2); and

— guidelines for the application of methods (from an external source).

7.4 Requirements and recommendations

7.4.1 To avoid systematic faults in the software architectural design and in the subsequent development activities, the description of the software architectural design shall address the following characteristics supported by appropriate guidelines and by notations for software architectural design as listed in Table 2:

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

a) unambiguity, consistency, comprehensibility, maintainability and verifiability of the architectural design;

b) modularity and abstraction; and

NOTE Abstraction can be supported by using hierarchical structures, grouping schemes or views to cover static, dynamic or deployment aspects of an architectural design.

c) low complexity.

Table 2 — Notations for software architectural design

Notations		ASIL			
		A	B	C	D
1a	Natural language [a]	++	++	++	++
1b	Informal notations	++	++	+	+
1c	Semi-formal notations [b]	+	+	++	++
1d	Formal notations	+	+	+	+

[a] Natural language can complement the use of notations for example where some topics are more readily expressed in natural language, or providing explanation and rationale for decisions captured in the notation

[b] Semi-formal notations include pseudocode or modelling with UML®, SysML™, Simulink® or Stateflow®, structured sentences or requirement patterns with controlled vocabulary

7.4.2 During the development of the software architectural design the following shall be considered:

a) the verifiability of the software architectural design;

NOTE This implies bi-directional traceability between the software architectural design and the software safety requirements.

b) the suitability for configurable software;

c) the feasibility for the design and implementation of the software units;

d) the testability of the software architecture during software integration testing; and

e) the maintainability of the software architectural design.

7.4.3 In order to avoid systematic faults, the software architectural design itself shall exhibit the following properties by use of the principles listed in Table 3:

a) comprehensibility;

b) modularity;

c) encapsulation; and

d) simplicity.

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

Table 3 — Principles for software architectural design

Principles		ASIL			
		A	B	C	D
1a	Appropriate hierarchical structure of the software components	++	++	++	++
1b	Restricted size and complexity of software components [a]	++	++	++	++
1c	Restricted size of interfaces [a]	+	+	+	++
1d	Strong cohesion within each software component [b]	+	++	++	++
1e	Loose coupling between software components [a, b, c]	+	++	++	++
1f	Appropriate scheduling properties	++	++	++	++
1g	Restricted use of interrupts [a]	+	+	+	++
1h	Use of only priority-based interrupts	+	+	+	++
1i	Appropriate management of shared resources [d]	++	++	++	++

[a] In principle 1b, 1c, 1e and 1g "restricted" means to minimize in balance with other design considerations.

[b] Principle 1d and 1e can, for example, be achieved by separation of concerns which refers to the ability to identify, encapsulate, and manipulate those parts of software that are relevant to a particular concept, goal, task, or purpose.

[c] Principle 1e addresses the limitation of dependencies between software components without an intended functional relationship.

[d] Applies for shared hardware resources as well as shared software resources. Such resource management can be implemented in software or hardware and includes mechanisms that prevents conflicting access to shared resources as well as mechanisms that detects conflicting access to shared resources and handle them.

NOTE 1 An appropriate compromise between the principles listed in Table 3 can be necessary since the principles are not mutually exclusive.

NOTE 2 Indicators for high complexity can be:

— highly branched control or data flow,

— excessive number of requirements allocated to single design elements,

— excessive number of interfaces of one design element or interactions between design elements,

— complex types or excessive number of parameters,

— excessive number of global variables,

— difficulties to provide evidence for suitability and completeness of error detection and handling,

— difficulties to achieve the required test coverage, or

— comprehensibility only for few experts or only by project participants.

NOTE 3 These properties and principles also apply to software routines (e.g. service routines for interrupt handling).

7.4.4 The software architectural design shall be developed down to the level where the software units are identified.

7.4.5 The software architectural design shall describe:

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

a) the static design aspects of the software architectural elements; and

NOTE 1 Static design aspects address:

— the software structure including its hierarchical levels;

— the data types and their characteristics;

— the external interfaces of the software components;

— the external interfaces of the software;

— the data flow at global variables; and

— the constraints including the scope of the architecture and external dependencies.

NOTE 2 In the case of model-based development, modelling the structure is an inherent part of the overall modelling activities.

b) the dynamic design aspects of the software architectural elements.

NOTE 1 Dynamic design aspects address:

— the functional chain of events and behaviour;

— the logical sequence of data processing;

— the control flow and concurrency of processes;

— the data flow between the software components;

— the data flow at external interfaces; and

— the temporal constraints.

NOTE 2 To determine the dynamic behaviour (e.g. of tasks, time slices and interrupts) the different operating states (e.g. power-up, shut-down, normal operation, calibration and diagnosis) are considered.

NOTE 3 To describe the dynamic behaviour (e.g. of tasks, time slices and interrupts) the communication relationships and their allocation to the system hardware (e.g. CPU and communication channels) are specified.

7.4.6 The software safety requirements shall be allocated to the software components. As a result, each software component shall be developed in compliance with the highest ASIL of any of the requirements allocated to it.

NOTE Following this allocation, further refinement of the software safety requirements can be necessary.

7.4.7 If an already existing software architectural element is used without modifications in order to meet the assigned safety requirements without being already developed according to ISO 26262, then it shall be qualified in accordance with ISO 26262-8:2018, Clause 12.

NOTE The use of qualified software components does not affect the applicability of Clauses 10 and 11. However, some activities described in Clauses 8 and 9 can be omitted.

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

NOTE The suitability for reuse of software elements already developed according to ISO 26262 is performed during the verification of the software architectural design.

7.4.8 If the embedded software has to implement software components of different ASILs, or safety-related and non-safety-related software components, then all of the embedded software shall be treated in accordance with the highest ASIL, unless the software components meet the criteria for coexistence in accordance with ISO 26262-9:2018, Clause 6.

7.4.9 If software partitioning (see Annex D) is used to implement freedom from interference between software components it shall be ensured that:

a) the shared resources are used in such a way that freedom from interference of software partitions is ensured;

NOTE 1 Tasks within a software partition are not free from interference among each other.

NOTE 2 One software partition cannot change the code or data of another software partition nor command non-shared resources of other software partitions.

NOTE 3 The service received from shared resources by one software partition cannot be affected by another software partition. This includes the performance of the resources concerned, as well as the rate, latency, jitter and duration of scheduled access to the resource.

b) the software partitioning is supported by dedicated hardware features or equivalent means (this requirement applies to ASIL D, in accordance with 4.4);

EXAMPLE Hardware feature such as a memory protection unit.

c) the part of the software that implements the software partitioning is developed in compliance with the highest ASIL assigned to any requirement of the software partitions; and

NOTE In general the operating system provides or supports software partitioning.

d) evidence for the effectiveness of the software partitioning is generated during software integration and testing (in accordance with Clause 10).

7.4.10 Safety analysis shall be carried out at the software architectural level in accordance with ISO 26262-9:2018, Clause 8, in order to:

— provide evidence for the suitability of the software to provide the specified safety-related functions and properties with the integrity as required by the respective ASIL;

NOTE Safety-related properties include independency and freedom from interference requirements.

— identify or confirm the safety-related parts of the software; and

— support the specification and verify the effectiveness of the safety mechanisms.

NOTE 1 Safety mechanisms can be specified to cover both issues associated with random hardware failures as well as software faults.

NOTE 2 See Annex E for additional information about the application of safety analyses at the software architectural level.

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

7.4.11 If the implementation of software safety requirements relies on freedom from interference or sufficient independence between software components, dependent failures and their effects shall be analysed in accordance with ISO 26262-9:2018, Clause 7.

NOTE See Annex E and ISO 26262-9:2018 Annex C for additional information about the application of analyses of dependent failures at the software architectural level.

7.4.12 To specify the necessary software safety mechanisms at the software architectural level, based on the results of the safety analysis at the software architectural level in accordance with 7.4.10 or 7.4.11, mechanisms for error detection as listed in Table 4 shall be applied.

Table 4 — Mechanisms for error detection at the software architectural level

Mechanisms		ASIL			
		A	B	C	D
1a	Range checks of input and output data	++	++	++	++
1b	Plausibility check [a]	+	+	+	++
1c	Detection of data errors [b]	+	++	++	++
1d	Monitoring of program execution [c]	o	+	++	++
1e	Temporal monitoring of program execution	o	+	++	++
1f	Diverse redundancy in the design	o	o	+	++
1g	Access permission control mechanisms [d]	+	+	++	++

[a] Plausibility checks can include using a reference model of the desired behaviour, assertion checks, or comparing signals from different sources.

[b] Types of mechanisms that may be used to detect data errors include error detecting codes and multiple data storage.

[c] An external element such as an ASIC or another software element performing a watchdog function. Monitoring can be logical or temporal monitoring or both.

[d] Control mechanisms may be implemented in software or hardware and are concerned with granting or denying access to safety-related shared resources

7.4.13 This subclause applies to ASIL (A), (B), C and D, in accordance with 4.4: to specify the necessary software safety mechanisms at the software architectural level, based on the results of the safety analysis at software architectural level in accordance with 7.4.10 and 7.4.11, mechanisms for error handling as listed in Table 5 shall be applied.

NOTE The analysis of possible hazards due to hardware is described in ISO 26262-5.

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

Table 5 — Mechanisms for error handling at the software architectural level

Mechanisms		ASIL			
		A	B	C	D
1a	Static recovery mechanism [a]	+	+	+	+
1b	Graceful degradation [b]	+	+	++	++
1c	Homogenous redundancy in the design [c]	+	+	+	++
1d	Diverse redundancy in the design [d]	o	o	+	++
1e	Correcting codes for data	+	+	+	+
1f	Inhibit access permission violations [e]	+	+	++	++

[a] Static recovery mechanisms can include the use of recovery blocks, backward recovery, forward recovery and recovery through repetition.

[b] Graceful degradation at the software level refers to prioritizing functions to minimize the adverse effects of potential failures on functional safety.

[c] Homogenous redundancy focuses primarily to control effects of transient faults or random faults in the hardware on which a similar software is executed (e.g. temporal redundant execution of software).

[d] Diverse redundancy implies dissimilar software in each parallel path and focuses primarily on the prevention or control of systematic faults in the software, common cause failures or common mode failures in the software or the hardware on which the software is executed.

[e] Access protection mechanisms may be implemented in software or hardware and are concerned with granting or denying access to safety-related shared resources

7.4.14 An upper estimation of required resources for the embedded software shall be made, including:

a) the execution time;

b) the storage space; and

EXAMPLE RAM for stacks and heaps, ROM for program and non-volatile data.

c) the communication resources.

7.4.15 The software architectural design shall be verified in accordance with ISO 26262-8:2018, Clause 9, and by using the software architectural design verification methods listed in Table 6 to provide evidence that the following objectives are achieved:

a) the software architectural design is suitable to satisfy the software requirements with the required integrity as stated by their ASIL;

b) the analyses of the software architectural design provide evidence for the suitability of the design;

c) compatibility with the target hardware; and

NOTE This includes the resources as specified in 7.4.14.

d) adherence to design guidelines.

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

Table 6 — Methods for the verification of the software architectural design

Methods		ASIL			
		A	B	C	D
1a	Walk-through of the design [a]	++	+	o	o
1b	Inspection of the design [a]	+	++	++	++
1c	Simulation of dynamic parts of the design [b]	+	+	+	++
1d	Prototype generation	o	o	+	++
1e	Formal verification	o	o	+	+
1f	Control flow analysis [c]	+	+	++	++
1g	Data flow analysis [c]	+	+	++	++
1h	Scheduling analysis	+	+	++	++

[a] In the case of model-based development these methods can also be applied to the model.
[b] Method 1c requires the usage of executable models for the dynamic parts of the software architecture.
[c] Control and data flow analysis can be limited to safety-related components and their interfaces.

7.5 Work products

7.5.1 Software architectural design specification resulting from requirements 7.4.1 to 7.4.14.

7.5.2 Software safety requirements specification (refined) resulting from requirement 7.4.6.

7.5.3 Safety analysis report resulting from requirement 7.4.10.

7.5.4 Dependent failures analysis report resulting from requirement 7.4.11.

7.5.5 Software verification report (refined) resulting from requirement 7.4.15.

8 Software unit design and implementation

8.1 Objectives

The objectives of this subphase are:

a) to specify the software units in accordance with the software architectural design and the associated software safety requirements; and

b) to implement the software units as specified.

8.2 General

Based on the software architectural design, the detailed design of the software units is developed. The detailed design can be represented in the form of a model. The source code can be manually or automatically generated from the design following coding or modelling guidelines. The functions and properties are achievable at the source code level if manual code development is used. If model-based development with automatic code generation is used, these properties apply at both model and code level, but can be verified only at model level provided that the code generation preserves these properties.

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

In order to develop a single software unit design both software safety requirements as well as all non-safety-related requirements are implemented. Hence in this subphase safety-related and non-safety-related requirements are handled within one development process.

8.3 Inputs to this clause

8.3.1 Prerequisites

The following information shall be available:

— design and coding guidelines for modelling and programming languages in accordance with 5.5.1;

— hardware-software interface specification (refined) in accordance with 6.5.2;

— software architectural design specification in accordance with 7.5.1; and

— software safety requirements specification (refined) in accordance with 7.5.2.

8.3.2 Further supporting information

The following information can be considered:

— technical safety concept (see ISO 26262-4:2018, 7.5.1);

— system architectural design specification (see ISO 26262-4:2018, 7.5.2);

— tool application guidelines (see 5.5.2);

— safety analysis report (see 7.5.3);

— dependent failure analysis report (see 7.5.4); and

— guidelines for the application of methods (from an external source).

8.4 Requirements and recommendations

8.4.1 The requirements of this subclause shall be complied with if the software unit is safety-related.

NOTE "Safety-related" means that the unit implements safety requirements, or that the criteria for coexistence (see ISO 26262-9:2018, Clause 6) of the unit with other units are not satisfied.

8.4.2 To avoid systematic faults and to ensure that the software unit design achieves the following properties, the software unit design shall be described using the notations listed in Table 7.

a) unambiguity;

b) consistency;

c) comprehensibility,;

d) maintainability; and

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

e) verifiability.

Table 7 — Notations for software unit design

Notations		ASIL			
		A	B	C	D
1a	Natural language [a]	++	++	++	++
1b	Informal notations	++	++	+	+
1c	Semi-formal notations [b]	+	+	++	++
1d	Formal notations	+	+	+	+

[a] Natural language can complement the use of notations for example where some topics are more readily expressed in natural language, or providing explanation and rationale for decisions captured in the notation.
EXAMPLE To avoid possible ambiguity of natural language when designing complex elements, a combination of an activity diagram with natural language can be used.
[b] Semi-formal notations include pseudocode or modelling with UML®, SysML™, Simulink® or Stateflow®, structured sentences or requirement patterns with controlled vocabulary.

NOTE In the case of model-based development with automatic code generation, the methods for representing the software unit design are applied to the model which serves as the basis for the code generation.

8.4.3 The specification of the software units shall describe the functional behaviour and the internal design to the level of detail necessary for their implementation.

EXAMPLE Internal design can include constraints on the use of registers and storage of data.

8.4.4 Design principles for software unit design and implementation at the source code level as listed in Table 8 shall be applied to achieve the following properties:

a) correct order of execution of subprograms and functions within the software units, based on the software architectural design;

b) consistency of the interfaces between the software units;

c) correctness of data flow and control flow between and within the software units;

d) simplicity;

e) readability and comprehensibility;

f) robustness;

EXAMPLE Methods to prevent implausible values, execution errors, division by zero, and errors in the data flow and control flow.

g) suitability for software modification; and

h) verifiability.

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

Table 8 — Design principles for software unit design and implementation

Principle		ASIL			
		A	B	C	D
1a	One entry and one exit point in subprograms and functions [a]	++	++	++	++
1b	No dynamic objects or variables, or else online test during their creation [a, b]	+	++	++	++
1c	Initialization of variables	++	++	++	++
1d	No multiple use of variable names [a]	++	++	++	++
1e	Avoid global variables or else justify their usage [a]	++	++	++	++
1f	Limited use of pointers [a]	+	++	++	++
1g	No implicit type conversions [a, b]	+	++	++	++
1h	No hidden data flow or control flow [c]	+	++	++	++
1i	No unconditional jumps [a, b, c]	++	++	++	++
1j	No recursions	+	+	++	++

[a] Principle 1a, 1b, 1d, 1e, 1f, 1g and 1i may not be applicable for graphical modelling notations used in model-based development.

[c] Principle 1h and 1i reduce the potential for modelling data flow and control flow through jumps or global variables.

NOTE For the C language, MISRA C[3] covers many of the principles listed in Table 8.

8.5 Work products

8.5.1 Software unit design specification resulting from requirements 8.4.2 to 8.4.4.

NOTE In the case of model-based development, the implementation model and supporting descriptive documentation, using methods listed in Table 8, specifies the software units.

8.5.2 Software unit implementation resulting from requirement 8.4.4.

9 Software unit verification

9.1 Objectives

The objective of this subphase is to provide evidence that the software detailed design and the implemented software units fulfil their requirements and do not contain undesired functionality.

9.2 General

The software detailed design and the implemented software units are verified by using an appropriate combination of verification measures such as reviews, analyses and testing.

9.3 Inputs to this clause

9.3.1 Prerequisites

The following information shall be available:

— hardware-software interface (HSI) specification (refined) in accordance with 6.5.2;

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

— software unit design specification in accordance with 8.5.1; and

— software unit implementation in accordance with 8.5.2.

9.3.2 Further supporting information

The following information can be considered:

— tool application guidelines (see 5.5.2); and

— guidelines for the application of methods (from an external source).

9.4 Requirements and recommendations

9.4.1 The requirements of this subclause shall be complied with if the software unit is safety-related.

NOTE "Safety-related" means that the unit implements safety requirements, or that the criteria for coexistence of the unit with other units are not satisfied.

9.4.2 Software unit verification shall be planned, specified and executed in accordance with ISO 26262-8:2018, Clause 9.

NOTE 1 Based on the definitions in ISO 26262-8:2018, Clause 9, the verification objects in the software unit verification are the software units.

NOTE 2 For model-based software development, the corresponding parts of the implementation model also represent objects for the verification planning. Depending on the selected software development process the verification objects can be the code derived from this model or the model itself.

9.4.3 The software unit design and the implemented software unit shall be verified by an appropriate combination of methods according to Table 9 to provide evidence for:

a) compliance with the software unit design specification (in accordance with Clause 8);

b) compliance with the specification of the hardware-software interface (in accordance with Clause 6), if applicable;

c) compliance with the software safety requirements allocated to the software units (in accordance with 7.4.6);

d) confidence in the absence of unintended functionality and properties;

e) robustness; and

EXAMPLE The effectiveness of error detection and error handling mechanisms in order achieve robustness of the software unit.

f) sufficient resources to support their functionality and properties.

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

Table 9 — Methods for software unit verification

Methods		ASIL			
		A	B	C	D
1a	Walk-through [a]	++	+	o	o
1b	Pair-programming	+	+	+	+
1c	Inspection [a]	+	++	++	++
1d	Semi-formal verification	+	+	++	++
1e	Formal verification	o	o	+	+
1f	Control flow analysis [b, c]	+	+	++	++
1g	Data flow analysis [b, c]	+	+	++	++
1h	Static code analysis [d]	++	++	++	++
1i	Static analyses based on abstract interpretation [e]	+	+	+	++
1j	Requirements-based test [f]	++	++	++	++
1k	Interface test	++	++	++	++
1l	Fault injection test [g]	+	+	+	++
1m	Resource usage test [h]	+	+	+	++
1n	Back-to-back comparison test between model and code, if applicable [i]	+	+	++	++

[a] For model-based development these methods are applied at the model level, if evidence is available that justifies confidence in the used code generator.

[b] Methods 1f and 1g can be applied at the source code level. These methods are applicable both to manual code development and to model-based development.

[c] Methods 1f and 1g can be part of methods 1e, 1h or 1i.

[d] Static analyses include MISRA rule checks and analyses of resource consumption.

[e] Method 1i is used for mathematical analysis of source code by use of an abstract representation of possible values for the variables. For this it is not necessary to translate and execute the source code.

[f] The software requirements at the unit level are the basis for this requirements-based test.

[g] In the context of software unit testing, fault injection test means to means to modify the tested software unit (e.g. introduce faults into the software) to test the robustness. Such modifications include injection of arbitrary faults (e.g. by corrupting values of variables, by introducing code mutations, or by corrupting values of CPU registers).

[h] Some aspects of the resource usage test can only be evaluated properly when the software unit tests are executed on the target hardware or if the emulator for the target processor supports resource usage tests.

[i] This method requires a model that can simulate the functionality of the software units. Here, the model and code are stimulated in the same way and results compared with each other.

EXAMPLE In the case of model-based design results of non-floating-point operations can be compared.

9.4.4 To enable the specification of appropriate test cases for the software unit testing in accordance with 9.4.3, test cases shall be derived using the methods as listed in Table 10.

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

Table 10 — Methods for deriving test cases for software unit testing

Methods		ASIL			
		A	B	C	D
1a	Analysis of requirements	++	++	++	++
1b	Generation and analysis of equivalence classes [a]	+	++	++	++
1c	Analysis of boundary values [b]	+	++	++	++
1d	Error guessing [c]	+	+	+	+

[a] Equivalence classes can be identified based on the division of inputs and outputs, such that a representative test value can be selected for each class.

[b] This method applies to interfaces, values approaching and crossing the boundaries and out of range values.

[c] Error guessing tests can be based on data collected through a "lessons learned" process and expert judgment.

9.4.5 To evaluate the completeness of verification and to provide evidence that the test objectives for unit testing are adequately achieved, the coverage of requirements at the software unit level shall be determined and the structural coverage shall be measured in accordance with the metrics as listed in Table 11. If the achieved structural coverage is considered insufficient, either additional test cases shall be specified or a rationale based on other methods shall be provided.

EXAMPLE 1 Analysis of structural coverage can reveal shortcomings in requirement-based test cases, inadequacies in requirements, dead code, deactivated code or unintended functionality.

EXAMPLE 2 A rationale can be given for the level of coverage achieved based on accepted dead code (e.g. code for debugging) or code segments depending on different software configurations; or code not covered can be verified using complementary methods (e.g. inspections).

EXAMPLE 3 A rationale can be based on state of the art and technology.

EXAMPLE 4 No target value or low target value for structural coverage without a rationale is considered insufficient.

Table 11 — Structural coverage metrics at the software unit level

Methods		ASIL			
		A	B	C	D
1a	Statement coverage	++	++	+	+
1b	Branch coverage	+	++	++	++
1c	MC/DC (Modified Condition/Decision Coverage)	+	+	+	++

NOTE 1 The structural coverage can be determined by the use of appropriate software tools.

NOTE 2 In the case of model-based development, the analysis of structural coverage can be performed at the model level using analogous structural coverage metrics for models.

EXAMPLE 5 The analysis of structural coverage performed at the model level may replace the source code coverage metrics if it is shown to be equivalent with rationales based on evidence that the coverage is representative of the code level.

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

NOTE 3 If instrumented code is used to determine the degree of coverage, it can be necessary to provide evidence that the instrumentation has no effect on the test results. This can be done by repeating the tests with non-instrumented code.

9.4.6 The test environment for software unit testing shall correspond as closely as possible to the target environment. If the software unit testing is not carried out in the target environment, the differences in the source and object code, and the differences between the test environment and the target environment, shall be analysed in order to specify additional tests in the target environment during the subsequent test phases.

NOTE 1 Differences between the test environment and the target environment can arise in the source code or object code, for example, due to different bit widths of data words and address words of the processors.

NOTE 2 Depending on the scope of the tests, the appropriate test environment for the execution of the software unit is used (e.g. the target processor, a processor emulator or a development system).

NOTE 3 Software unit testing can be executed in different environments, for example:

— model-in-the-loop tests,

— software-in-the-loop tests,

— processor-in-the-loop tests, or

— hardware-in-the-loop tests.

NOTE 4 For model-based development, software unit testing can be carried out at the model level followed by back-to-back comparison tests between the model and the object code. The back-to-back comparison tests are used to ensure that the behaviour of the models with regard to the test objectives is equivalent to the automatically-generated code.

9.5 Work products

9.5.1 Software verification specification resulting from requirements 9.4.2 to 9.4.6.

9.5.2 Software verification report (refined) resulting from requirement 9.4.2.

10 Software integration and verification

10.1 Objectives

The objectives of this subphase are:

a) to integrate the software elements; and

b) to provide evidence that the embedded software complies with the software architectural design.

10.2 General

In this subphase, the particular integration levels and the interfaces between the software elements are verified against the software architectural design. The steps of the integration and verification of the software elements correspond directly to the hierarchical architecture of the software.

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

The embedded software can consist of safety-related and non-safety-related software elements.

10.3 Inputs to this clause

10.3.1 Prerequisites

The following information shall be available:

— hardware-software interface (HSI) specification (refined) in accordance with 6.5.2;

— software architectural design specification in accordance with 7.5.1;

— software unit implementation in accordance with 8.5.2; and

— software verification specification in accordance with 9.5.1.

10.3.2 Further supporting information

The following information can be considered:

— qualified software components available (see ISO 26262-8:2018, Clause 12);

— software tool qualification report (see ISO 26262-8:2018, 11.5.2);

— tool application guidelines (see 5.5.2); and

— guidelines for the application of methods (from an external source).

10.4 Requirements and recommendations

10.4.1 The planning of the software integration shall describe the steps for integrating the individual software units hierarchically into software components until the embedded software is fully integrated, and shall consider:

a) the functional dependencies that are relevant for software integration; and

b) the dependencies between the software integration and the hardware-software integration.

NOTE For model-based development, the software integration can be replaced with integration at the model level and subsequent automatic code generation from the integrated model.

10.4.2 Software integration verification shall be planned, specified and executed in accordance with ISO 26262-8:2018, Clause 9.

NOTE 1 Based on the definitions in ISO 26262-8:2018, Clause 9, the software integration verification objects are the software components.

NOTE 2 For model-based development, the verification objects can be the models associated with the software components.

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

10.4.3 The software integration shall be verified by an appropriate combination of methods according to Table 12 to provide evidence that both the software components and the embedded software achieve:

a) compliance with the software architectural design in accordance with Clause 7;

b) compliance with the hardware-software interface specification in accordance with 6.5.2;

c) the specified functionality;

d) the specified properties;

EXAMPLE Reliability due to absence of inaccessible software, robustness against erroneous inputs, dependability due to effective error detection and handling.

e) sufficient resources to support the functionality; and

f) effectiveness of the software partitioning , if applicable.

Table 12 — Methods for verification of software integration

Methods		ASIL			
		A	B	C	D
1a	Requirements-based test [a]	++	++	++	++
1b	Interface test	++	++	++	++
1c	Fault injection test [b]	+	+	++	++
1d	Resource usage test [c, d]	++	++	++	++
1e	Back-to-back comparison test between model and code, if applicable [e]	+	+	++	++
1f	Analyses of the control or data flow	+	+	++	++
1g	Static code analysis [f]	++	++	++	++
1h	Static analyses based on abstract interpretation [g]	+	+	+	+

[a] The software requirements at the architectural level are the basis for this requirements-based test.

[b] In the context of software integration testing, fault injection test means to introduce faults into the software for the purposes described in Clause 10.4.3 and in particular to test the correctness of hardware-software interface related to safety mechanisms. This includes injection of arbitrary faults in order to test safety mechanisms (e.g. by corrupting software interfaces). Fault injection can also be used to verify freedom from interference.

[c] To ensure the fulfilment of requirements influenced by the hardware architectural design with sufficient tolerance, properties such as average and maximum processor performance, minimum or maximum execution times, storage usage (e.g. RAM for stack and heap, ROM for program and data) and the bandwidth of communication links (e.g. data buses) have to be determined.

[d] Some aspects of the resource usage test can only be evaluated properly when the software integration tests are executed on the target hardware or if the emulator for the target processor supports resource usage tests.

[e] This method requires a model that can simulate the functionality of the software components. Here, the model and code are stimulated in the same way and results compared with each other.

[f] Static analyses include modelling or coding rule checks (e.g. MISRA) and analyses of resource consumption.

[g] Abstract interpretation of the source code, analyses of pointers, invalid uses of locks.

NOTE 2 For model-based development, the verification objects can be the models associated with the software components.

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

10.4.4 To enable the specification of appropriate test cases for the software integration test methods selected in accordance with 10.4.3, test cases shall be derived using the methods as listed in Table 13.

Table 13 — Methods for deriving test cases for software integration testing

Methods		ASIL			
		A	B	C	D
1a	Analysis of requirements	++	++	++	++
1b	Generation and analysis of equivalence classes [a]	+	++	++	++
1c	Analysis of boundary values [b]	+	++	++	++
1d	Error guessing [c]	+	+	+	+

[a] Equivalence classes can be identified based on the division of inputs and outputs, such that a representative test value can be selected for each class.

[b] This method applies to parameters or variables, values approaching and crossing the boundaries and out of range values.

[c] Error guessing tests can be based on data collected through a "lessons learned" process and expert judgment.

10.4.5 To evaluate the completeness of verification and to provide evidence that the test objectives for integration testing are adequately achieved, the coverage of requirements at the software architectural level by test cases shall be determined. If necessary, additional test cases shall be specified or a rationale based on other methods shall be provided.

10.4.6 This subclause applies to ASIL (A), (B), C and D, in accordance with 4.4: To evaluate the completeness of test cases and to provide evidence that the test objectives for integration testing are adequately achieved, the structural coverage shall be evaluated in accordance with the methods as listed in Table 14. If the achieved structural coverage is considered insufficient, either additional test cases shall be specified or a rationale based on other methods shall be provided.

EXAMPLE Analysis of structural coverage can reveal shortcomings in the requirement-based test cases, inadequacies in the requirements, dead code, deactivated code or unintended functionality.

Table 14 — Structural coverage at the software architectural level

Methods		ASIL			
		A	B	C	D
1a	Function coverage [a]	+	+	++	++
1b	Call coverage [b]	+	+	++	++

[a] Method 1a refers to the percentage of executed software functions. This evidence can be achieved by an appropriate software integration strategy.

[b] Method 1b refers to the percentage of executed software function calls.

NOTE 1 The structural coverage can be determined using appropriate software tools.

NOTE 2 In the case of model-based development, software architecture testing can be performed at the model level using analogous structural coverage metrics for models.

10.4.7 It shall be verified that the embedded software that is to be included as part of a production release in accordance with ISO 26262-2:2018, Clause 6.4.10, contains all the specified functions and

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

properties, and only contains other unspecified functions if these functions do not impair the compliance with the software safety requirements.

EXAMPLE In this context unspecified functions include code used for debugging or instrumentation.

NOTE If deactivation of these unspecified functions can be ensured, this is an acceptable means of compliance with this requirement. Otherwise the removal of such code is a change (see ISO 26262-8:2018, Clause 8).

10.4.8 The test environment for software integration testing shall correspond as closely as possible to the target environment. If the software integration testing is not carried out in the target environment, the differences in the source and object code and the differences between the test environment and the target environment shall be analysed in order to specify additional tests in the target environment during the subsequent test phases.

NOTE 1 Differences between the test environment and the target environment can arise in the source or object code, for example, due to different bit widths of data words and address words of the processors.

NOTE 2 Depending on the scope of the tests and the hierarchical level of integration, the appropriate test environments for the execution of the software elements are used. Such test environments can be the target processor for final integration, or a processor emulator or a development system for the previous integration steps.

NOTE 3 Software integration testing can be executed in different environments, for example:

— model-in-the-loop tests,

— software-in-the-loop tests,

— processor-in-the-loop tests, or

— hardware-in-the-loop tests.

10.5 Work products

10.5.1 Software verification specification (refined) resulting from requirements 10.4.1 to 10.4.8.

10.5.2 Embedded software resulting from requirement 10.4.1.

10.5.3 Software verification report (refined) resulting from requirement 10.4.2.

11 Testing of the embedded software

11.1 Objective

The objective of this subphase is to provide evidence that the embedded software fulfils the software safety requirements.

11.2 General

The purpose of this activity is to provide evidence that the embedded software fulfils its requirements in the target environment.

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

11.3 Inputs to this clause

11.3.1 Prerequisites

The following information shall be available:

— software architectural design specification in accordance with 7.5.1;

— software safety requirements specification (refined) in accordance with 7.5.2; and

— software verification specification (refined) in accordance with 10.5.1.

11.3.2 Further supporting information

The following information can be considered:

— technical safety concept (see ISO 26262-4:2018, 7.5.1);

— system architectural design specification (see ISO 26262-4:2018, 7.5.2);

— integration and verification report (see ISO 26262-4:2018, 8.5.2);

— tool application guidelines (see 5.5.2); and

— guidelines for the application of methods (from an external source).

11.4 Requirements and recommendations

11.4.1 To verify that the embedded software fulfils the software safety requirements, tests shall be conducted in the test environments as listed in Table 15.

NOTE Test cases that already exist, for example from software integration testing, can be re-used.

Table 15 — Test environments for conducting the software testing

Methods		ASIL			
		A	B	C	D
1a	Hardware-in-the-loop	++	++	++	++
1b	Electronic control unit network environments [a]	++	++	++	++
1c	Vehicles	+	+	++	++

[a] Examples include test benches partially or fully integrating the electrical systems of a vehicle, “lab-cars” or “mule” vehicles, and “rest of the bus” simulations.

11.4.2 The software test cases derived by methods as listed in Table 16 shall be executed to provide evidence that the embedded software fulfils the software requirements as required by their respective ASIL.

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

Table 16 — Methods for deriving test cases for the test of the embedded software

Methods		ASIL			
		A	B	C	D
1a	Analysis of requirements	++	++	++	++
1b	Generation and analysis of equivalence classes	+	++	++	++
1c	Analysis of boundary values	+	+	++	++
1d	Error guessing based on knowledge or experience	+	+	++	++
1e	Analysis of functional dependencies	+	+	++	++
1f	Analysis of operational use cases	+	++	++	++
1g	Fault injection test [a]	+	+	+	++

[a] In the context of software testing, fault injection test means to introduce faults into the software by means of e.g. corrupting calibration parameters.

11.4.3 The results of the verification of the software safety requirements shall be evaluated with regard to:

a) compliance with the expected results;

b) coverage of the software safety requirements; and

NOTE this includes the coverage of the configuration and calibration ranges. See Annex C.

c) pass or fail criteria.

11.5 Work products

11.5.1 Software verification specification (refined) resulting from requirements 11.4.1 to 11.4.3.

11.5.2 Software verification report (refined) resulting from requirements 11.4.1 to 11.4.3.

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

Annex A (informative) Overview of and workflow of management of product development at the software level

Table A.1 provides an overview of objectives, prerequisites and work products of the particular phases of the product development at the software level.

Table 17 — Overview of product development at the software level

Clause	Objectives	Prerequisites	Work products
5 General topics for the product development at the software level	The objective of this clause is to give an overview of the product development at the software level and to provide topics specific to software development.	(none)	5.5.1 Design and coding guidelines for modelling and programming languages 5.5.2 Tool application guidelines
6 Specification of software safety requirements	The objectives of this subphase are: a) to specify the software safety requirements. They are derived from the technical safety concept and the system design specification; b) to refine the hardware-software interface requirements initiated in ISO 26262-4:2018, Clause 7; and c) to verify that the software safety requirements and the hardware-software interface requirements are consistent with the technical safety concept and the system design specification.	Technical safety concept (see ISO 26262-4:2018, 7.5.1) System architectural design specification (see ISO 26262-4:2018, 7.5.2) Hardware-software interface (HSI) specification (see ISO 26262-4:2018, 7.5.6)	6.5.1 Software safety requirements specification 6.5.2 Hardware-software interface (HSI) specification (refined) 6.5.3 Software verification report
7 Software architectural design	The objective of this subphase are: a) to develop a software architectural design that satisfies the software safety requirements and the non-safety-related requirements; b) to verify that the software architectural design is suitable to satisfy the software safety requirements with the required ASIL; and c) to support the implementation of the software.	Design and coding guidelines for modelling and programming languages (see 5.5.1) Hardware-software interface (HSI) specification (refined) (see 6.5.2) Software safety requirements specification (see 6.5.1)	7.5.1 Software architectural design specification 7.5.2 Software safety requirements specification (refined) 7.5.3 Safety analysis report 7.5.4 Dependent failures analysis report 7.5.5 Software verification report (refined)

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

Table A.1 (*continued*)

Clause	Objectives	Prerequisites	Work products
8 Software unit design and implement-ation	The objectives of this subphase are: a) to specify the software units in accordance with the software architectural design and the associated software safety requirements; and b) to implement the software units as specified.	Design and coding guidelines for modelling and programming languages (see 5.5.1) Hardware-software interface (HSI) specification (refined) (see 6.5.2) Software architectural design specification (see 7.5.1) Software safety requirements specification (refined) (see 7.5.2)	8.5.1 Software unit design specification 8.5.2 Software unit implementation
9 Software unit verification	The objective of this subphase is to provide evidence that the software detailed design and the implemented software units fulfil their requirements and do not contain undesired functionality.	Hardware-software interface (HSI) specification (refined) (see 6.5.2) Software unit design specification (see 8.5.1) Software unit implementation (see 8.5.2)	9.5.1 Software verification specification 9.5.2 Software verification report (refined)
10 Software integration and verification	The objectives of this subphase are: a) to integrate the software elements; and b) to provide evidence that the embedded software complies with the software architectural design.	Hardware-software interface (HSI) specification (refined) (6.5.2) Software architectural design specification (see 7.5.1) Software unit implementation (see 8.5.2) Software verification specification (see 9.5.1) Software verification report (refined) (see 9.5.2)	10.5.1 Software verification specification (refined) 10.5.2 Embedded software 10.5.3 Software verification report (refined)
11 Testing of the embedded software	The objective of this subphase is to provide evidence that the embedded software fulfils the software safety requirements.	Software architectural design specification (see 7.5.1) Software safety requirements specification (refined) (see 7.5.2) Software verification specification (refined) (see 10.5.2)	11.5.1 Software verification specification (refined) 11.5.2 Software verification report (refined)
Annex C Software config-uration	The objective of software configuration is to enable controlled changes in the behaviour of the software for different applications	See applicable prerequisites of the relevant phases of the safety lifecycle in which software configuration is applied.	C.5.1 Configuration data specification C.5.2 Calibration data specification C.5.3 Configuration data C.5.4 Calibration data C.5.5 Software verification plan (refined) C.5.6 Verification specification (refined) C.5.7 Verification report (refined)

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

Annex B (informative) Model-based development approaches

B.1 Objectives

This annex explains possible usage benefits and risks of model-based development approaches (MBDV) during the development at the software level.

NOTE 1 Risks mentioned in this annex can potentially cause the violation of safety requirements and safety goals.

NOTE 2 This annex does not imply that the model-based development approaches mentioned are restricted to software development only.

B.2 General

This section provides general information that is common to various use cases of MBDV (e.g. requirements, design, verification/testing). Section B.3 provides specific considerations based on exemplary use cases.

B.2.1 Introduction

Models may be used to represent information or views of information required during the development phase as well as for the abstract design and implementation of one or more software elements or the software's environment. Models themselves may consist of various hierarchical refinement levels or references to refined models at a lower hierarchical level (e.g. hierarchical structure with black box and white box views for each model element). A model is developed using a modelling notation.

Modelling notations may be graphical and/or textual. They may be formal (e.g. notation with an underlying mathematical definition) or semi-formal (e.g. structured notation with incompletely defined semantics). Modelling notations may be international standards (e.g. UML) or company-specific. They are typically based on concepts such as classes, block diagrams, control diagrams, or state charts (expressing states and transition between states). In this annex, only models and modelling notations are considered which incorporate a defined syntax and semantics adequate for their intended use (i.e. illustrative figures without such definitions are not considered as models). Beside the specific reasons for using MBDV (e.g. simulation or code generation), an adequately defined syntax and semantics is a basis for the achievement of criteria such as comprehensibility, unambiguity, correctness, consistency and verifiability of information or work products described by models especially when different parties are collaborating.

In addition to the modelling notation itself, modelling guidelines and/or suitable tools are means to achieve adequately defined syntax and semantics.

EXAMPLE As part of guidelines naming conventions or style guides support the achievement of criteria such as "comprehensibility" while the selection of language subsets excluding ambiguous constructs supports both "comprehensibility" and the correct transformation and execution of the model. In case of simulation or transformation of models suitable simulation engines or code generators can supplement the required semantics such as a deterministic order of execution or transformation.

In this annex the following use cases of model-based development approaches are considered further:

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

— Specification of software safety requirements (Clause 6 and ISO 26262-8:2018, Clause 6);

— Development of the software architectural design (Clause 7);

— Design and implementation of software units, with or without automated code generation (Clause 8);

— Design, implementation and integration of software components, including automated code generation from software component models (Clauses 8, 9 and 10); and

— Verification (static and/or dynamic) (Clauses 9, 10 and 11).

B.2.2 General suitability aspects

The suitability of the selected model-based development approach for an intended application, including the modelling techniques, notations, languages or tools, can be evaluated based on aspects such as:

— achievement of criteria provided by ISO 26262 for the corresponding development phase (e.g. comprehensibility, completeness, verifiability, unambiguity, accuracy) with the selected approach(es),

— knowledge and experiences with the selected approach(es),

— adequacy of the definition of syntax and semantics,

— adequacy for the application domain (e.g. real-time behaviour, data structures, generation of software for a fixed-point versus floating-point microcontroller),

— role of the model in the software lifecycle (e.g. development of safety requirements, expressing static or dynamic aspects of architectural designs),

— support for managing complexity by enforcing the principles of abstraction, hierarchy and modularity,

— collaboration model between internal/external development partners (e.g. consistency of data during data exchange),

— required versus available confidence in software tools (see ISO 26262-8:2018, Clause 11), or

— qualification of software components (e.g. software libraries) used as part of selected approach or included into selected tools (see ISO 26262-8:2018, Clause 12).

B.2.3 Potential impact of MBDV on the software lifecycle

A model may represent more than one work product of ISO 26262-6 (e.g. requirements, architectural design, detailed design and model-based software integration with code generation). In comparison to a traditional development process where lifecycle data are separated, a stronger coalescence of the phases “Software safety requirements”, “Software architectural design”, “Software unit design and implementation” etc. may occur. The potential benefits of this approach (e.g. continuity, information

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

sharing across the software life cycle, consistency) are appealing, but this approach may also introduce risks (see clause B.3).

B.3 Specific considerations for the example software development use cases

This section provides considerations related to benefits and risks based on example use cases.

B.3.1 Specification of software safety requirements (Clause 6 and ISO 26262-8:2018, Clause 6)

Models can be used to capture the functionality, behaviour, design or properties of the software to be realized at safety requirements level (e.g. open/closed loop controls, controlled systems, states and transitions, monitoring functions or independence properties) and to enable verification and validation of requirements by simulation or formal methods.

Models are a suitable means to achieve the characteristics of safety requirements and to satisfy requirements to use semi-formal or formal notations as stated in ISO 26262-8, Clause 6.

EXAMPLE When specifying a state machine using a model based approach, the corresponding model captures the required states and transitions in an intuitive, unambiguous and comprehensible way. The fact that such a model represents more than one atomic requirement compared with a textual specification of the same state machine is acceptable. An adequate rationale would be that for this purpose completeness, verifiability and internal consistency of the specification can be achieved and maintained better by the model without duplication of information.

However be aware that by using requirement models it is possible to miss safety requirements that cannot be expressed by the selected model-based development approach. For this reason ISO 26262-8, Clause 6 emphasizes an appropriate combination of textual requirements and other notations.

EXAMPLE Requirements related to the robustness of the software functionality against missing or erroneous inputs

B.3.2 Development of the software architectural design (Clause 7)

Models can be used to capture the software architecture according to several perspectives, in particular:

a) Static aspects (e.g. components, their interfaces and data flows, components attributes); and

b) Dynamic aspects (e.g. functional chain of events, scheduling, message passing).

Models together with the modelling guidelines are a suitable mean to achieve the properties of and principles for safety-related architectural designs and to satisfy requirements for semi-formal or formal notations as stated in Clause 7.

However be aware that one modelling approach might not express all of the required aspects (e.g. models targeting the implementation of software functions often do not describe the overall architectural design including elements of the basic software or operating system) and that a model

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

may not be fully understandable without further textual explanations (e.g. by different collaborating parties).

B.3.3 Design and implementation of software units, with or without automated code generation (Clauses 8 and 9)

Models together with the modelling and coding guidelines can be used to develop the detailed design and the software unit itself at a higher level of abstraction. Depending on the selected approach the software unit can be implemented by code generation from the model.

This approach enables automated consistency checks (e.g. data typing, correct definition of data before use) and verification by formal methods, simulation, static analyses or model-in-the-loop/software-in-the-loop testing of the software units. Depending on the confidence in the code generator automated code generation from the model may lower the risk of coding errors compared to manual coding.

However be aware that for example:

— Design aspects that cannot be expressed by the modelling notations might be missed, if not documented in some other appropriate form. Even unit testing (e.g. using back-to-back-testing) might not reveal this, if all tests are only derived from such an “incomplete” model.

— Safety-related code properties such as robustness might not be implemented, if only the intended function is modelled. Code properties often are achieved by an appropriate combination of the model properties and code generation properties (e.g. if the division is generated “as is” by the code generator, then avoidance of division by zero will be checked at model level. If the code generator handles the division by zero with a robust design coding pattern, division by zero analysis in the model may be alleviated).

— Incorrect addressing or handling of numeric accuracy aspects might introduce faults (e.g. if the model is just inherited from control engineering). For instance issues resulting from different precision properties of the control engineer’s computer run time environment and the target microcontroller run time environment.

— The creation of discrete models of the software behaviour from continuous models may introduce systematic faults (e.g. effect of discretization of values and/or time, execution time of the calculation, handling concurrent tasks).

— Simulation or verification results (e.g. results of consistency checks) may be tool-dependent because different simulation engines or model checkers may deviate regarding the implemented semantics (e.g. for algorithmic aspects not addressed by the semantics of the modelling language).

— The resource usage in case of auto-coding (execution time and memory consumption) may be higher than in case of manual coding.

— In case of auto-coding verifiability of the generated source code equivalence to manual code may be more difficult to achieve.

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

B.3.4 Design, implementation and integration of software components, including automated code generation from software component models (Clauses 8, 9 and 10)

Models together with the modelling and coding guidelines can be used to design, implement and integrate software at a higher level of abstraction. Thus software components or even the embedded software can be implemented by code generation from the integrated model.

This approach implies a strong coalescence of the different software development phases and therefore the benefits versus risks of such an approach need to be evaluated carefully (see hints provided above) including the implementation of appropriate safety measures (e.g. using separate test models for deriving additional test cases).

B.3.5 Verification (static and/or dynamic) (Clauses 9, 10 and 11)

Models can serve as a mean for verifying work products (e.g. design model, code). Such a model is called a "verification model". Verification models can be used for example as the basis for back-to-back testing, generation of test cases or for expressing safety-related properties as a reference for formal verification (this requires that the model to verify is formal too).

Models can also serve as a means to enable or support verification activities (e.g. plant model needed for the hardware-in-the-loop testing of closed-loop controls by simulating the environment of the device under test).

The use of models for verification purposes may enable the early and efficient detection of faults/failures in work products or software, more efficient test case generation, highly automated testing or even formal verification techniques.

However be aware that for example:

— Without evidence for the suitability of a model used for verification purposes the validity of related results might be questionable (e.g. inadequate plant models may lead to incorrect test results).

— For valid results the maturity of the verification model must match the target maturity of the device under test to be verified.

— Test cases generated from the same model which is also used for code generation cannot serve as the only source for verifying both the model itself and the code generated from it.

Annex C (normative) Software configuration

C.1 Objectives

The objective of software configuration is to enable controlled changes in the behaviour of the software for different applications.

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

C.2 General

Configurable software enables the development of application specific software using configuration and calibration data (see Figure C.1).

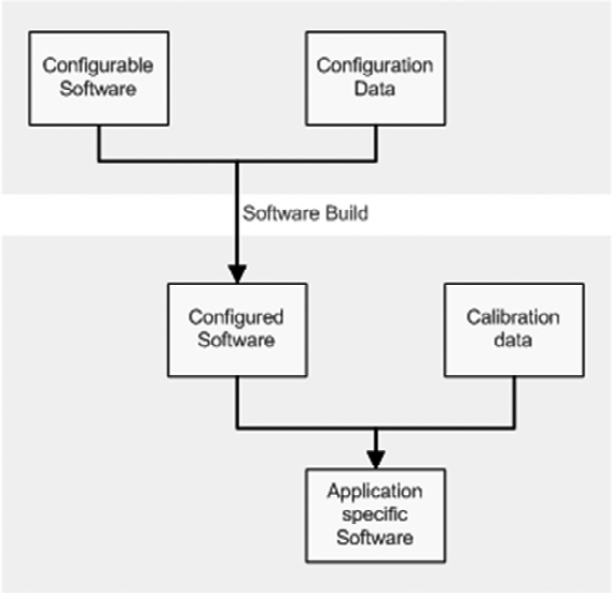


Figure 2 — Creating application specific software

C.3 Inputs to this clause

C.3.1 Prerequisites

The prerequisites are in accordance with the relevant phases in which software configuration is applied.

C.3.2 Further supporting information

See applicable further supporting information of the relevant phases in which software configuration is applied.

C.4 Requirements and recommendations

C.4.1 The configuration data shall be specified to ensure the correct usage of the configurable software during the safety lifecycle. This shall include:

a) the valid values of the configuration data;

b) the intent and usage of the configuration data;

c) the range, scaling, units; and

d) the interdependencies between different elements of the configuration data.

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

C.4.2 Verification of the configuration data shall be planned and performed in accordance with ISO 26262-8:2018, Clause 9 to ensure:

a) the use of values within their specified range; and

b) the compatibility with the other configuration data.

NOTE The testing of the configured software is performed within the test phases of the software lifecycle (see Clause 9, Clause 10, Clause 11 and ISO 26262-4:2018, Clause 8).

C.4.3 The ASIL of the configuration data shall equal the highest ASIL of the configurable software by which it is used.

C.4.4 The verification of configurable software shall be planned, specified and executed in accordance with ISO 26262-8:2018, Clause 9. Configurable software shall be verified for the configuration data set that is to be used for the item development under consideration.

NOTE Only that part of the embedded software whose behaviour depends on the configuration data is verified against the configuration data set.

C.4.5 For configurable software a simplified software safety lifecycle in accordance with Figures C.2 or C.3 may be applied.

NOTE A combination of the following verification activities can achieve the complete verification of the configured software:

a) “verification of the configurable software”,

b) “verification of the configuration data”, and

c) “verification of the configured software”.

This is achieved by either

— verifying a range of admissible configuration data in a) and showing compliance to this range in b), or

— by showing compliance to the range of admissible configuration data in b) and performing c).

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

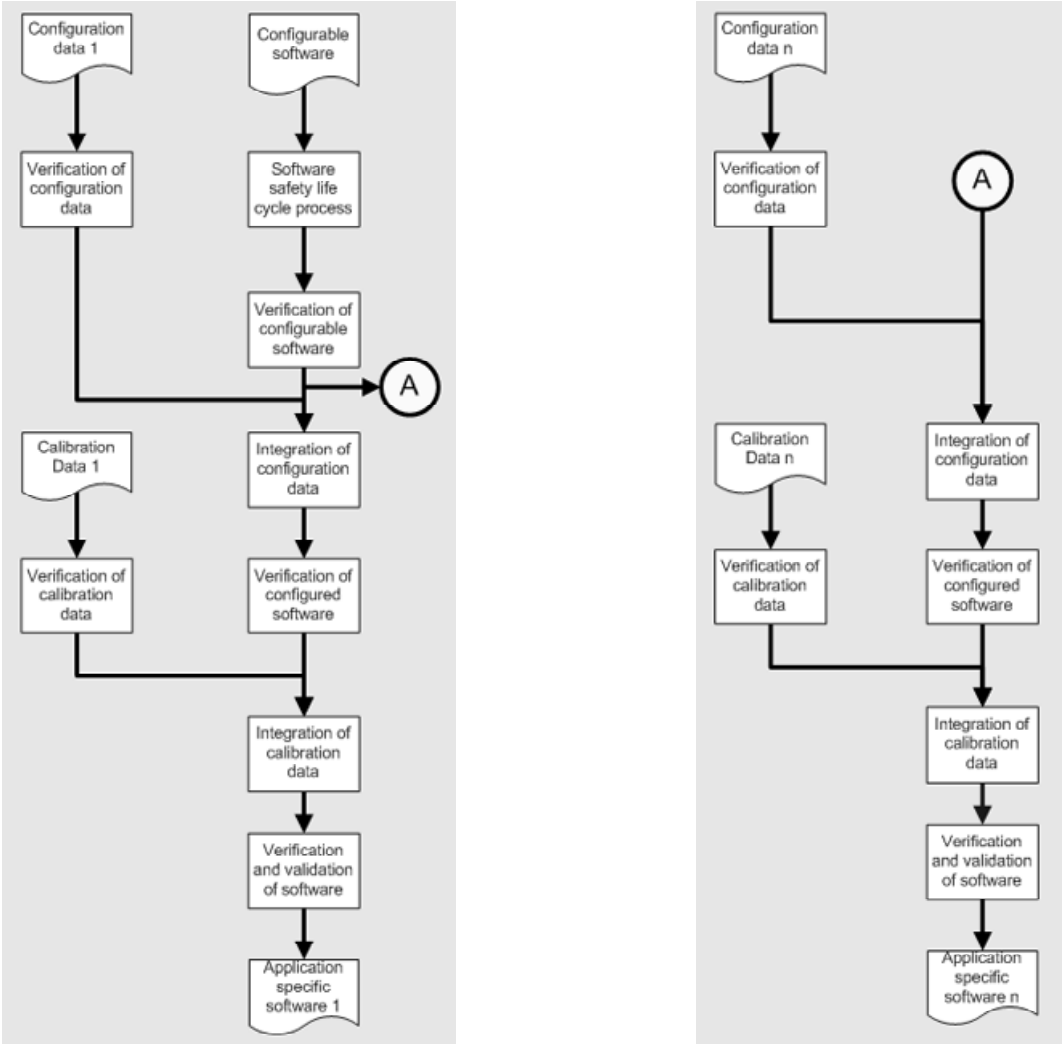


Figure 3 — Variants of the reference phase model for software development with configurable software and different configuration data and different calibration data

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

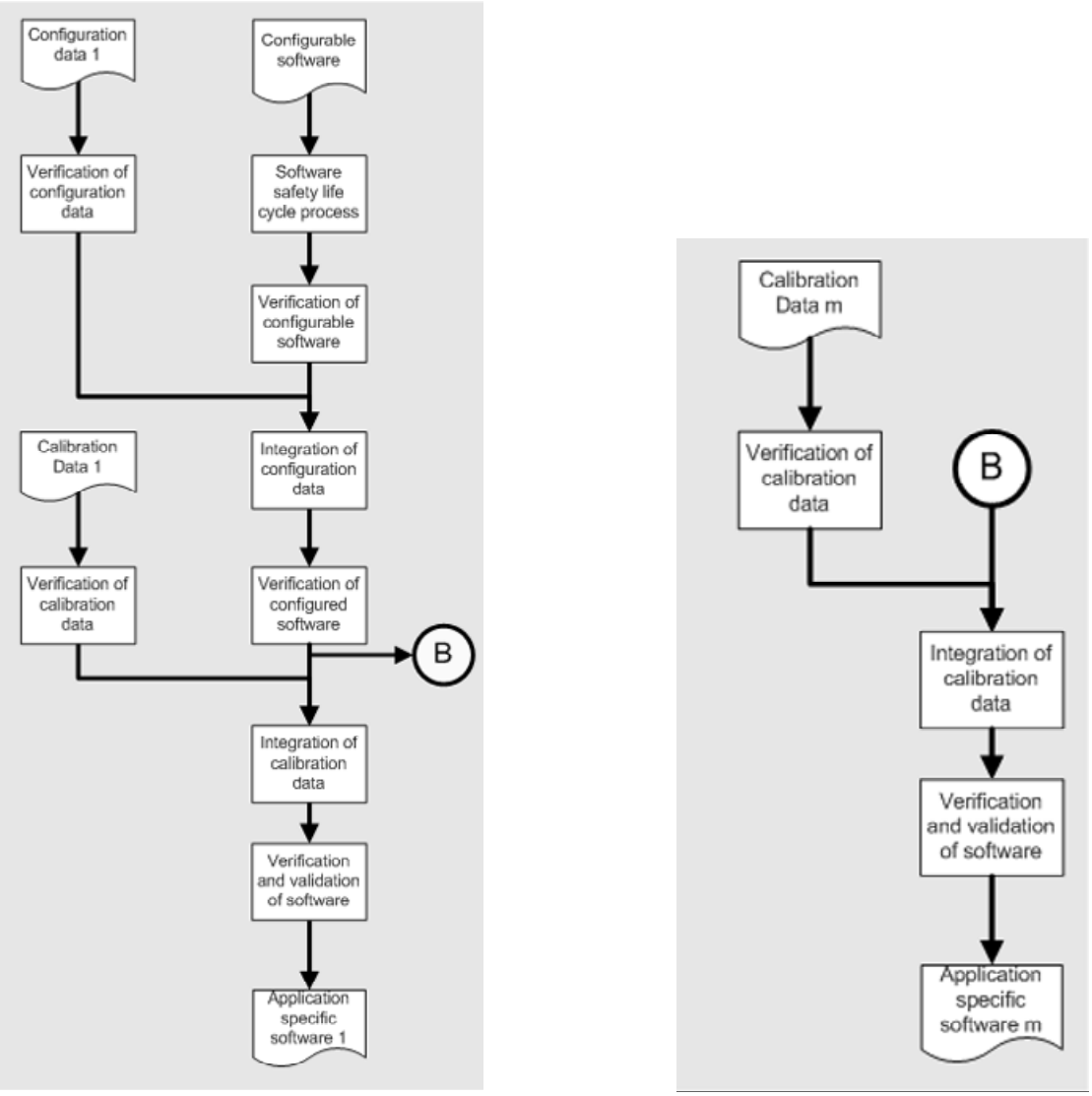


Figure 4 — Variants of the reference phase model for software development with configurable software with the same configuration data and different calibration data

C.4.6 The calibration data associated with software components shall be specified to ensure the correct operation and expected performance of the configured software. This shall include:

a) the valid values of the calibration data;

b) the intent and usage of the calibration data;

c) the range, scaling and units, if applicable, with their dependence on the operating state;

d) the known interdependencies between different calibration data; and

NOTE Interdependencies can exist between calibration data within one calibration data set or between calibration data in different calibration data sets such as those applied to related functions implemented in the software of separate ECUs.

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

e) the known interdependencies between configuration data and calibration data.

NOTE Configuration data can have an impact on the configured software that uses the calibration data.

C.4.7 The verification of the calibration data shall be planned, specified and executed in accordance with ISO 26262-8:2018, Clause 9. The verification of calibration data shall examine whether the calibration data is within its specified boundaries.

NOTE Verification of calibration data can also be performed within application-specific software verification, or at runtime by the configurable software.

C.4.8 The ASIL of the calibration data shall equal the highest ASIL of the software safety requirements it can violate.

C.4.9 To detect unintended changes of safety-related calibration data, mechanisms for the detection of unintended changes of data as listed in Table C.1 shall be applied.

Table 18 — Mechanisms for the detection of unintended changes of data

Mechanisms		ASIL			
		A	B	C	D
1a	Plausibility checks on calibration data	++	++	++	++
1b	Redundant storage and comparison of calibration data	+	+	+	++
1c	Calibration data checks using error detecting codes [a]	+	+	+	++

[a] Error detecting codes may also be implemented in the hardware in accordance with ISO 26262-5:2018.

C.4.10 The planning of the generation and application of calibration data shall specify:

a) the procedures that shall be followed;

b) the tools for generating calibration data; and

c) the procedures for verifying calibration data.

NOTE Verification of calibration data can include checking the value ranges of calibration data or the interdependencies between different calibration data.

C.5 Work products

C.5.1 Configuration data specification resulting from requirements C.4.1 and C.4.3.

C.5.2 Calibration data specification resulting from requirement C.4.6.

C.5.3 Configuration data resulting from requirement C.4.3.

C.5.4 Calibration data resulting from requirement C.4.8.

C.5.5 Software verification plan (refined) resulting from requirements C.4.2, C.4.4, C.4.7 and C.4.10.

C.5.6 Verification specification (refined) resulting from requirements C.4.4 and C.4.7.

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

C.5.7 Verification report (refined) resulting from requirements C.4.1, C.4.4, C.4.7 and C.4.8.

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

Annex D (informative) Freedom from interference between software elements

D.1 Objectives

The objective is to provide examples of faults that can cause interference between software elements (e.g. software elements of different software partitions). Additionally, this Annex provides examples of possible mechanisms that can be considered for the prevention, or detection and mitigation of the listed faults.

NOTE The capability and effectiveness of the mechanisms used to prevent, or to detect and mitigate relevant faults is assessed during development.

D.2 General

D.2.1 Achievement of freedom from interference

To develop or evaluate the achievement of freedom from interference between software elements, the effects of the exemplary faults, and the propagation of the possible resulting failures can be considered.

D.2.2 Timing and execution

With respect to timing constraints, the effects of faults such as those listed below can be considered for the software elements executed in each software partition:

— blocking of execution,

— deadlocks,

— livelocks,

— incorrect allocation of execution time, or

— incorrect synchronization between software elements.

EXAMPLE Mechanisms such as cyclic execution scheduling, fixed priority based scheduling, time triggered scheduling, monitoring of processor execution time, program sequence monitoring and arrival rate monitoring can be considered.

D.2.3 Memory

With respect to memory, the effects of faults such as those listed below can be considered for software elements executed in each software partition:

— corruption of content,

— inconsistent data (e.g. due to update during data fetch),

— stack overflow or underflow, or

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

— read or write access to memory allocated to another software element.

EXAMPLE 1 Safety measures such as memory protection, parity bits, error-correcting code (ECC), cyclic redundancy check (CRC), redundant storage, restricted access to memory, static analysis of memory accessing software and static allocation can be used.

EXAMPLE 2 Appropriate verification methods can be considered as detailed safety analysis to identify critical memories that protection mechanisms are used for. Results of static and semantic code analysis, control flow and data flow analysis can provide evidence for demonstrating freedom from interference.

D.2.4 Exchange of information

With respect to the exchange of information, the causes for faults or effects of faults such as those listed below can be considered for each sender or each receiver:

— repetition of information,

— loss of information,

— delay of information,

— insertion of information,

— masquerade or incorrect addressing of information,

— incorrect sequence of information,

— corruption of information,

— asymmetric information sent from a sender to multiple receivers,

— information from a sender received by only a subset of the receivers, or

— blocking access to a communication channel.

NOTE The exchange of information between elements executed in different software partitions or different ECUs includes signals, data, messages, etc.

EXAMPLE 1 Information can be exchanged using I/O-devices, data busses, etc.

EXAMPLE 2 Mechanisms such as communication protocols, information repetition, loop back of information, acknowledgement of information, appropriate configuration of I/O pins, separated point-to-point unidirectional communication objects, unambiguous bidirectional communication objects, asynchronous data communication, synchronous data communication, event-triggered data buses, event-triggered data buses with time-triggered access, time-triggered data busses, mini-slotting and bus arbitration by priority can be used.

EXAMPLE 3 Communication protocols can contain information such as identifiers for communication objects, keep alive messages, alive counters, sequence numbers, error detection codes and error-correcting codes.

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

EXAMPLE 4 Appropriate verification methods can be considered as detailed safety analysis to identify critical data exchange that protection mechanisms are used for. Results of static and semantic code analysis, control flow and data flow analysis can provide evidence for demonstrating freedom from interference.

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

Annex E (informative) Application of safety analyses and analyses of dependent failures at the software architectural level

E.1 Objectives

This Annex explains the possible application of safety analyses and dependent failure analyses at the software architectural level. The examples provided in this Annex are intended to support the understanding of these analyses and provide guidance on their application.

E.2 General

E.2.1 Scope and purpose of the analyses

The ability of the embedded software to provide the specified functions, behaviour and properties with the integrity as required by the assigned ASIL is also examined or confirmed by applying safety analyses or/and dependent failure analyses at the software architectural level.

The suitability of embedded software to provide the specified functions and properties with the integrity required by the assigned ASIL is examined by:

— identifying possible design weaknesses, conditions, faults or/and failures that could induce causal chains leading to the violation of safety requirements (e.g. using an inductive or deductive method);

— analysing the consequences of possible faults, failures or/and causal chains on the functions and properties required for the software architectural elements or/and the embedded software.

The achieved level of independence or freedom from interference between the relevant software architectural elements is examined by analysing the software architectural design with respect to dependent failures, i.e.:

— possible single events, faults or/and failures that may cause a malfunctioning behaviour of more than one of the software elements which require independence from each other (e.g. cascading and/or common cause failures and/or common mode failures);

— possible single events, faults or/and failures that may propagate from one software element to another inducing causal chains leading to the violation of safety requirements (e.g. cascading failures).

Achievement of independence or freedom from interference between the software architectural elements can be required because of:

— the application of an ASIL decomposition at the software level (see 6.4.3),

— the application of a design approach that combines architectural elements with different assigned ASIL and/or elements with assigned ASIL and QM elements in a single software architectural design (see 7.4.6, 7.4.8 and 7.4.9), or

— the implementation of a specific functional behaviour or specific properties.

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

The role of safety analyses and analyses of dependent failures at software architectural level supports both design specification and design verification activities.

Such analysis may also reveal incompleteness or inconsistencies with respect to given safety requirements. Therefore, consistency between existing safety requirements and safety measures, and newly identified requirements or measures, is beneficial.

The results of safety analyses and analyses of dependent failures at software architectural level is also a basis for

— the specification and implementation of effective safety mechanisms in the product, or

— the determination of appropriate development-time safety measures, which are suitable either to prevent or to detect and control the relevant faults or failures identified during these analyses.

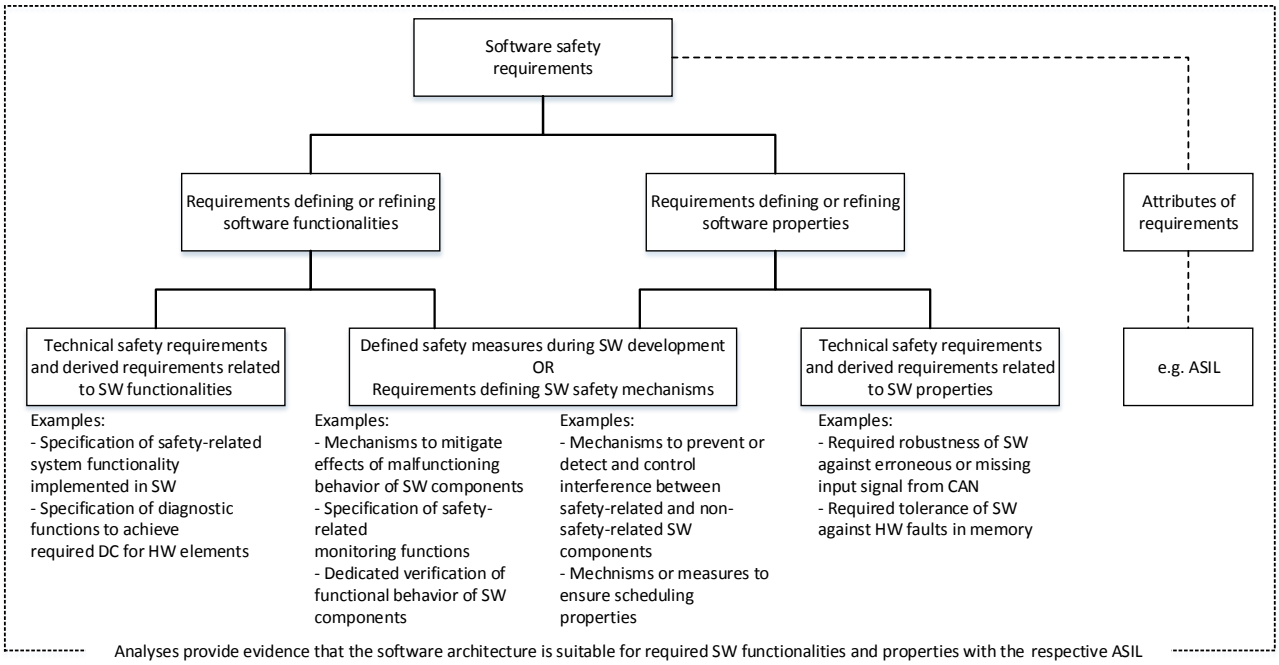


Figure E.1 — Relationship between software safety requirements and safety analyses

To explain such a relationship Figure E.2 illustrates interference caused by a conflicting use of a shared resource (e.g. a shared processing element). In this example the QM software element interferes and prevents the timely execution of the ASIL software element. Examples are shown both without and with implementation of freedom from interference.

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

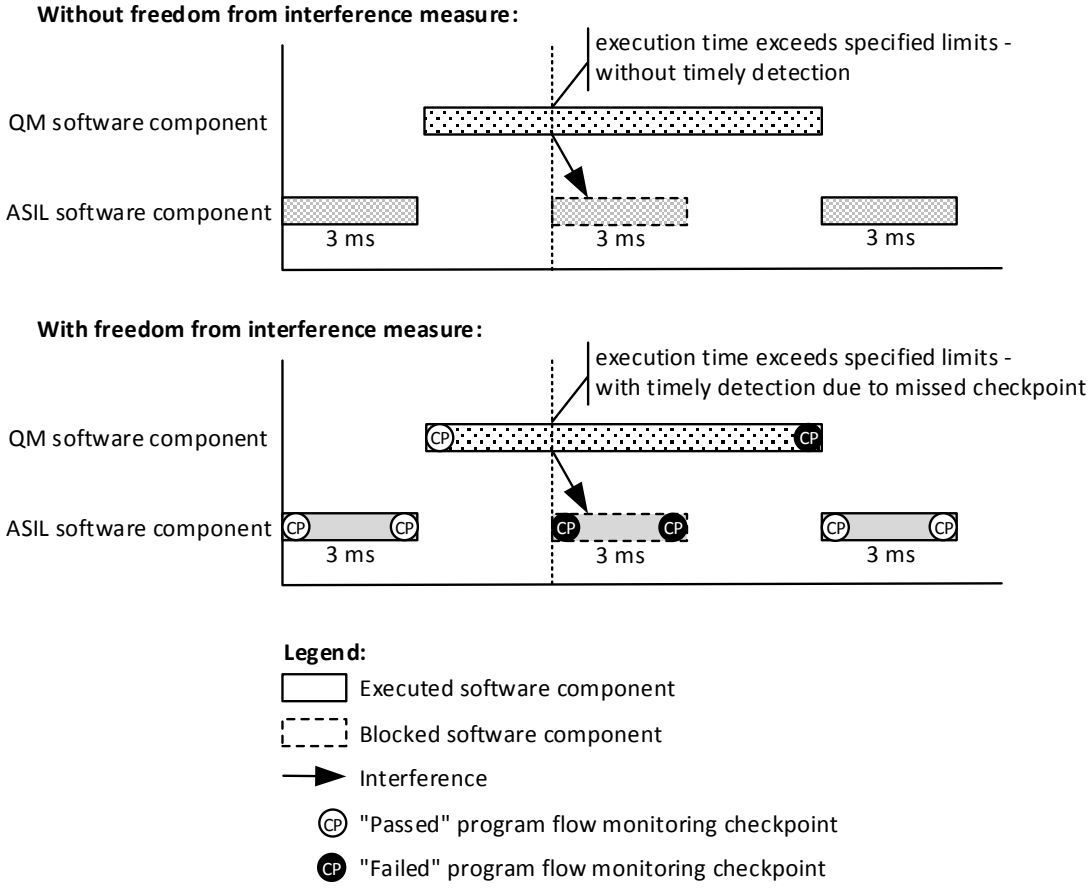


Figure E.2 — Example of timing interference leading to a cascading failure

Safety analyses and dependent failure analyses are intended to be applied at the software architecture level. Code level safety analyses (e.g. with respect to runtime errors such as divide by zero, access beyond array index boundary) are neither required nor are they considered appropriate for the following reasons:

— The residual risk regarding these types of faults can be considered as sufficiently low provided that the software development approach as described in ISO 26262-6:2018 including the selection and application of suitable methods, approaches and development principles such as modelling and programming guidelines, design and implementation rules, model and/or code reviews, semantic or static analysis, unit and integration tests is applied carefully;

— These types of faults in the implementation of the respective architectural elements usually induce the same malfunctioning behaviour of the element at its interfaces as analysed at architectural level. Therefore the analyses at the higher level already provide the evidence that the selected technical safety mechanisms are effective or that the development-time safety measures are sufficient to ensure an adequate argument.

Exceptional cases may exist in which a code level analysis may be appropriate because of a specific situation, a specific software safety concept or a specific software argument, but a decision to perform such analyses usually are related to weaknesses in architectural design concepts.

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

E.2.2 Relationship between safety analyses and dependent failure analyses at the system, hardware and software level

Safety analyses and dependent failure analyses are applied during the different phases of the development lifecycle with possible dependencies and interactions.

In a separation of concern approach initiated at the system level it is usually the responsibility of hardware development to ensure that the processing unit possesses a sufficient low residual risk regarding hardware faults. Therefore the software safety analysis and software dependent failure analysis at architectural level can be performed without considering hardware faults.

However under some conditions (e.g. insufficient diagnostic coverage regarding transient faults or random hardware faults of the processing element) it may be appropriate to consider the negative influence of hardware faults. In such cases specific analyses regarding the specific hardware/software interaction are performed considering for example:

— The specific software mapping to the different processing units of multi-core systems;

— The detailed design of the processing element on which the software is executed regarding the static or dynamic aspects of the software architectural design;

— The achievable diagnostic coverage of the selected software safety mechanisms regarding software-relevant hardware faults.

NOTE Analysis of a specific hardware/software interaction can be supported by fault injection techniques (see also ISO 26262-11:2018 Clause 4.8).

E.2.3 Safety analyses in a distributed software development (including SEooC development of software elements)

Embedded software and its software elements are often developed as a distributed development, by more than one organization (including OEM, Tier 1, Tier 2, silicon vendor or software vendor) and even out of context (e.g. basic software, hardware-related software, COTS software or operating systems developed as SEooC).

Important aspects for meaningful analyses in distributed software developments are:

— Definition and agreement about the scope of the analyses and the procedures or methods used for performing the analyses either separately or jointly (including exchange of information/documentation or confirmation of assumptions);

— Definition and agreement about the interfaces when using a modular approach for the analyses (e.g. agreed software fault models at the interfaces between the different scopes, the approach used or the exchanged information/documents);

— Definition and agreement about the verification of the analyses (e.g. joint reviews).

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

E.3 Topics for performing analyses

The software architecture design is an important means to manage the complexity of the software development by defining and providing the design aspects (e.g. using more formalized or less formalized notations) needed to develop and integrate the software. From the safety perspective, it is important to summarize the appropriate topics and levels of detail in such way that the achievement of functional safety can be argued.

Common topics to achieve meaningful analyses are the determination of:

— the specific goals for the analyses (e.g. derive them from required functionality/properties of software such as robustness or deterministic execution) – see E.3.1;

— the scope and boundaries regarding the specific architectural design to be analysed (including interfaces) – see E.3.2;

— the method applied to conduct the analyses – see E.3.3;

— means or criteria to support the significance, objectivity and rigour of the analyses – see E.3.4.

Further guidance on these topics and possible dependencies is provided in the following paragraphs.

E.3.1. Determination of the specific goals for the analyses

Besides the general goals provided by Clause 7 and ISO 26262-9, specific goals for the analyses can be derived from the safety requirements assigned to the software or to software components.

EXAMPLE Application-specific unwanted events or malfunctioning behaviour caused by the violation of the safety-related functionalities or properties assigned to the software within the scope of the analyses.

E.3.2 Determination of the scope and boundaries regarding the specific architectural design to be analysed

Depending on the goals of the analyses the scope is determined setting the focus to the relevant parts of the architectural design.

The scope of the analyses can be influenced by:

— the relevant scope of delivery and the respective responsibilities as defined in the DIA,

— the specific goals of the analyses,

— architectural strategies supported by “good” design principles (see 7.4.3), or by

— properties required from the architectural design resulting from higher level safety concepts in respect of the achievement of freedom from interference or sufficient independence.

EXAMPLE 1 Appropriate end-to-end data protection mechanisms can be used as an argument that the basic software can treated as a “black box” when considering the exchange of safety-related data with external senders or receivers during a safety analyses.

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

EXAMPLE 2 Dependencies of software functions with operating modes such as a safety-related function which is only "active" in a mode where the other "critical" function is safely deactivated by a safe mode switching function.

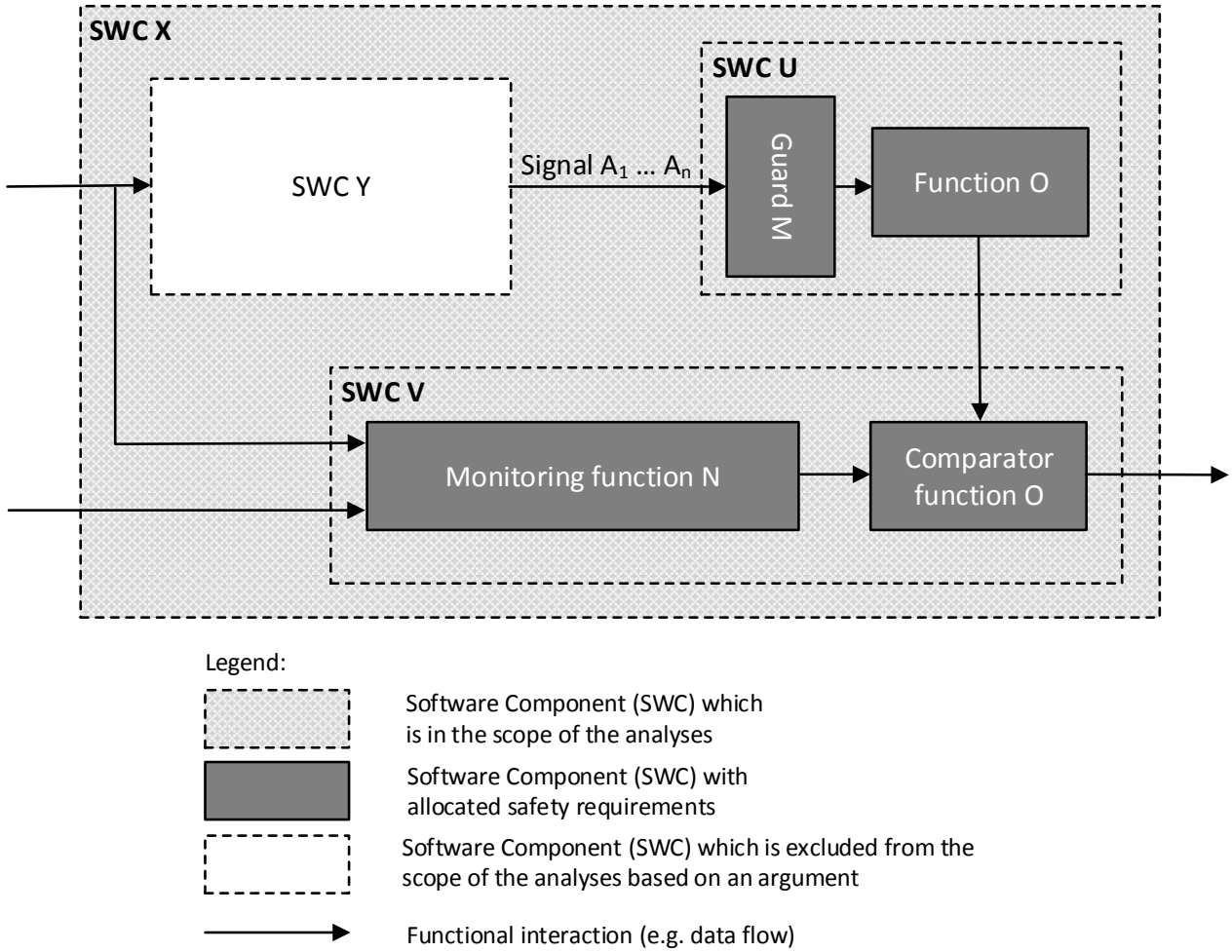


Figure E.3 — Scope of the analyses with respect to the safety argumentation

Figure E.3 shows an example where according to the safety concept of SWC X the safety-related functionalities are implemented only in SWC U and controlled by a monitoring function implemented in SWC V. Determination of the scope of the analyses on the architecture clarifies the required safety arguments and supports the further steps shown in the following sections. In this example, SWC Y can be exempted from a detailed analysis unless the analyses of SWC V and SWC U is unable to confirm the assumed freedom-from-interference (e.g. because of weaknesses in the Guard M regarding cascading failures with respect to erroneous signals A_1 to A_n).

E3.3. Determination of the method applied to conduct the analyses

Due to the specific nature of software (e.g. no random faults due to wear out or ageing and lack of a mature probabilistic method) methods established for such analyses at the system or hardware level often cannot be transferred to software without modifications or will not deliver results that are as meaningful.

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

Common factors of safety analyses methods are that they usually consist of a means to enforce a structured approach, to store the gained knowledge and to draw conclusions.

EXAMPLE Syntax and semantics to describe faults and their dependencies in a fault tree or the function net or list of elements or Risk Priority Number used in an FMEA.

The basis for the safety analyses and dependent failure analyses is the architectural design describing the static aspects (e.g. expressed by a block diagram showing the functional elements and their interfaces/relationships) as well as the dynamic aspects (e.g. expressed by sequence, timing or scheduling diagrams or state charts) of the related software.

The safety analyses and dependent failure analyses according to the architectural design can follow functional and/or processing chains considering both static and dynamic aspects. During such analyses software fault models or questions such as "which specific fault or failure originating from or impacting this software component can lead to the violation of an assigned safety requirement" can be used to identify safety-related weaknesses in the design.

Refinements of architectural design can support the safety analyses and dependent failure analyses for the adequate level of details in order to identify lower levels of faults or failures.

The level of detail required during such an analyses is related to:

— adequate selection of the scope during refinements of the architecture design,

— properties that support to achieve the specific goals of the analyses,

— the argument based on the architectural strategy supported by "good" design principles (see 7.4.3), or

— the argument resulting from achieved freedom from interference or sufficient independence.

E.3.4 Aspects to support the meaningfulness, objectivity and rigour of the analyses

E.3.4.1 Dependent failure initiators for DFA in accordance with ISO 26262-9:2018

In Annex C of ISO 26262–9:2018 dependent failure initiators are defined that can be used to improve the results of a dependent failure analysis performed on functional or processing chains of the software that are required to be independent. In practice, at the software level the following amplifications for the dependent failure initiators can be used for guiding the analyses (e.g. a checklist).

Table E.1 – Analysis at the software level

Type of independence violation		Examples for the software level
Shared resource	Same hardware element instance used by the two functions which are therefore affected by the failure or unavailability of that shared resource, i.e. from a	• Software component used by more than one other software component, e.g. – reused standard software modules – libraries – middleware

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

	physical perspective.	– basic software – operating system including scheduler – communication stack – parameter file – execution time – allocated memory
Components of identical type	Identical components may fail with higher probability simultaneously (common mode failures)	• Reused standard software modules • Standard library components • Middleware • Operating system • Communication stack • Parameter set
Communication between the two functions	Receiving function is affected by information that is false, lost, sent multiple times, or in the wrong order etc. from the sender	• Information passed via argument through a function call, or via writing/reading a variable being global to the two software functions (data flow) • Data or message corruption/ repetition*/ loss*/ delay*/ masquerading or incorrect addressing of information* • Insertion*/ sequence of information* • Corruption of information* • Asymmetric information sent from a sender to multiple receivers* • Information from a sender received by only a subset of the receivers* • Blocking access to a communication channel*
Shared information inputs	Same information consumed by the two functions even in absence of shared resources, i.e. from a functional perspective	• Calibration parameters • Constants, or variables, being global to the two software functions • Basic software passes data (read from hardware register and converted into logical information) to two application software functions • Data / function parameter arguments / messages delivered by software function to more than one other function *See also "Communication between the two functions"*
Environmental influences	Common external environmental disturbance affects both functions	n.a.
Unintended impact	Two functions affecting each other's elements directly via an implicit, that is unintended,	• Memory / corruption of content* – "wild pointers" (including

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

	interface.	program counter stack) – memory mis-allocation – memory leaks – read/write access to memory allocated to another software element* – buffer under-/overflow • Timing and execution – deadlocks* – livelocks* – incorrect allocation of execution time* – incorrect synchronization between software elements*
Systematic coupling	Systematic causes from human or tool errors can lead to the simultaneous failure of more than one function	• Manufacturing fault / repair fault (e.g. false flashing, false calibration reference for sensors) • Non-diverse development approaches including: – same software tools (e.g. IDE, compiler, linker) – same algorithms – same programming and/or modelling language used – same complier/linker used • Same personnel • Same social-cultural context (even if different personnel) • Development fault, e.g. – human error – insufficiently qualified personnel – process weaknesses – insufficient methods
* These examples are taken from Annex D on freedom from interference between software elements. In that respect, the dependent failure initiators Unintended Impact and Communication represent causes of violation of freedom from interference for software.		

E.3.4.2 Use content of Annex D

Annex D provides software fault models to be considered when analysing interference between software elements, for example with respect to:

— timing and execution,

— memory, or

— exchange of information.

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

E.3.2 Analyses supported by using guide words

Guide words can be used to systematically examine the possible deviations from a specific design intent and to determine their possible consequences. Guide words are used to generate questions for the examination of a specific functionality or property of the architectural element during the analyses. When using guide words the analyses of the specific functions or properties for each design element are repeated with each guide word until the predetermined guide words have all been examined. Thus guide words are a mean to conduct such analyses systematically and to provide an argument for completeness.

Data flow or similar diagrams are often used to describe the functional aspects of the software architectural design (e.g. functional or processing chains). Figure E.4 shows interaction of the software components Y, U and V and the related functional or processing chain (e.g. based on data flow).

Guide words can be used to identify weaknesses, faults, and failures. The selection of suitable guide words depends on the characteristics of the examined functions, behaviour, properties, interfaces or exchanged data.

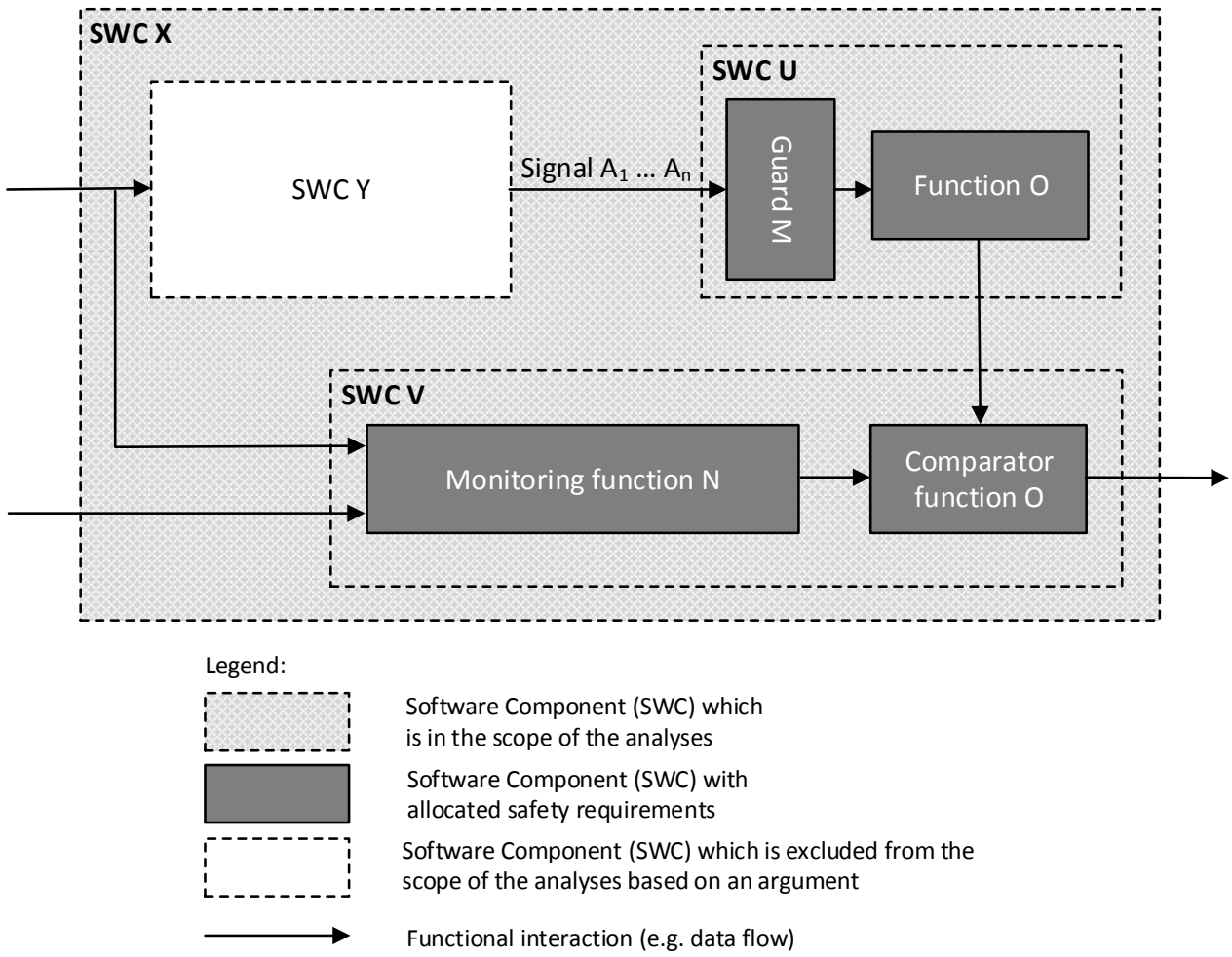


Figure E.4 — Block diagram of software architectural design

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

Using the design intent as a basis the guide words are derived by combining design attributes, the related guide words and their interpretation in order to achieve a common understanding for the analyses. Table E.2 shows examples for the selection of guide words.

Table E.2 – Example for the selection of guide words

Examined functionality or property	Guide word	Interpretation	Additional references
signal flow (A_1....A_n)	After / Late	Signal too late or out of sequence.	D.2.4 Delay of information D.2.4 Incorrect sequence of information D.2.4 Blocking access to communication channel D.2.2 Blocking of execution D.2.2 Deadlocks D.2.2 Livelocks D.2.2 Incorrect allocation of execution time D.2.2 Incorrect synchronization between software components Message was sent or received or queued too late The initial message was lost and the communication subsystem attempted a re-try without notifying the application Sending or receiving process was blocked too long
	Before / Early	Signal too early or out of sequence.	D.2.4 Incorrect sequence of information D.2.2 Incorrect synchronization between software components Message sent too early Sending process scheduled too often
	No	No signal	D.2.2 Blocking of execution D.2.2 Deadlocks D.2.2 Livelocks D.2.2 Incorrect synchronization between software components
Signal value	More	Signal value exceeds permitted range	
	Less	Signal value falls bellows the permitted range	

During the analysis guide words can be applied in a deductive or inductive approach. During the analyses a further refinement of the guide words is possible if required (e.g. to better match specific design aspects).

During the analyses the guide words are used according to the selected method (e.g. following function or processing chains) to generate the questions that will examine the design and reveal possible weaknesses and their consequences. An example for a guide word supported analysis of the Signals A_1 … A_n is shown in Table E.3.

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

Table E.3 – Example for a guide word supported analysis

Guide word	Interpretation	Consequence	Safety measure	Required activity
More	Signal value exceeds permitted range	Function O will generate erroneous output	Guard M limits Signal $A_1 \ldots A_n$ to the permitted maximum range	Implement Guard M
Less	Signal value falls bellows the permitted range	Function O will generate erroneous output	Guard M limits Signal $A_1 \ldots A_n$ to the permitted maximum range	Implement Guard M
Other than	Values of signals $A_1 \ldots A_n$ are inconsistent	Function O will generate erroneous output	Guard M checks $A_1 \ldots A_n$ for consistency using physical dependencies	Implement Guard M

E.4 Mitigation strategy for weaknesses identified during safety and dependent failure analyses

The result of safety analyses and dependent failure analyses at the software architectural level allow the selection of adequate safety measures to prevent or to detect and control faults and failures with the potential to violate safety requirements or safety goals. Figure E.5 shows a possible strategy for the determination of suitable safety measures based on such results.

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

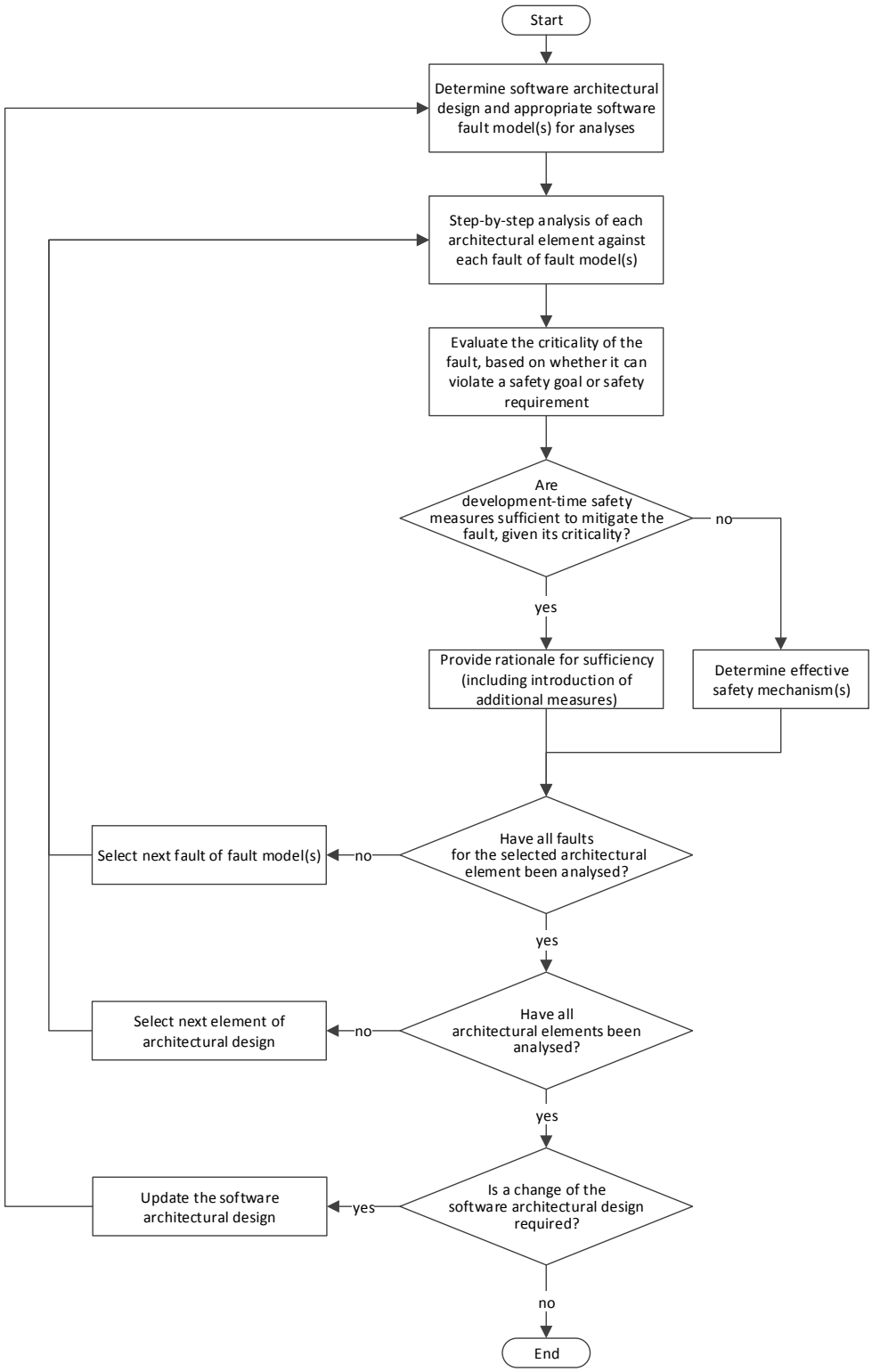


Figure E.5 — Decision graph for "acceptance" of counter measures (i.e. technical on-line safety mechanisms vs. development-time safety measures)

As shown in Figure E.5 it is not always necessary to implement safety mechanisms which are able to prevent or detect and control such faults or failures at runtime.

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

Choosing a suitable mitigation strategy can be based on aspects such as:

— complexity of the software architectural design (e.g. number of interfaces and software components) and/or the software components (e.g. number of assigned requirements or size of the component), or

EXAMPLE For more complex software an argument based solely on development-time measures is less suitable.

— effectiveness of development-time safety measures in order to prevent the relevant faults or failures (e.g. effectiveness of verification activities considering the software fault models used during the analyses).

© ISO 2016 – All rights reserved

Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.

Bibliography

[1] ISO/IEC 12207, *Systems and software engineering — Software life cycle processes*

[2] IEC 61508 (all parts), *Functional safety of electrical/electronic/programmable electronic safety-related systems*

[3] MISRA C:2012, *Guidelines for the use of the C language in critical systems*, ISBN 978-1-906400-10-1, MIRA, March 2013

[4] MISRA AC AGC, *Guidelines for the application of MISRA-C:2004 in the context of automatic code generation*, ISBN 978-1-906400-02-6, MIRA, November 2007

[5] Bieman, J.M.; Dreilinger, D.; Lijun Lin, "Using fault injection to increase software test coverage," in Software Reliability Engineering, 1996. Proceedings., Seventh International Symposium on , vol., no., pp.166-174, 30 Oct-2 Nov 1996 doi: 10.1109/ISSRE.1996.558776

[6] Jia, Y., Merayo, M., and Harman, M. (2015) Introduction to the special issue on Mutation Testing. Softw. Test. Verif. Reliab., 25: 461–463.

[7] ISO 26262-12:2018, *Road Vehicles — Functional safety— Part 12: Adaptation of ISO 26262 for Motorcycles*

© ISO 2016 – All rights reserved
Licensed to MDS Technology CO.,LTD. / Seungyong Han (seungyong@mdstec.com)
ISO Store Order: OP-186844 / Downloaded: 2016-12-26
Single user licence only, copying and networking prohibited.